

Simulation-based Optimization of User Interfaces for Quality-assuring Machine Learning Model Predictions

YU ZHANG, University of Oxford, The United Kingdom

MARTIJN TENNEKES and TIM DE JONG, Statistics Netherlands, The Netherlands

LYANA CURIER, Open University of the Netherlands, The Netherlands

BOB COECKE and MIN CHEN, University of Oxford, The United Kingdom

Quality-sensitive applications of machine learning (ML) require quality assurance (QA) by humans before the predictions of an ML model can be deployed. QA for ML (QA4ML) interfaces require users to view a large amount of data and perform many interactions to correct errors made by the ML model. An optimized user interface (UI) can significantly reduce interaction costs. While UI optimization can be informed by user studies evaluating design options, this approach is not scalable, because there are typically numerous small variations that can affect the efficiency of a QA4ML interface. Hence, we propose using simulation to evaluate and aid the optimization of QA4ML interfaces. In particular, we focus on simulating the combined effects of human intelligence in initiating appropriate interaction commands and machine intelligence in providing algorithmic assistance for accelerating QA4ML processes. As QA4ML is usually labor-intensive, we use the simulated task completion time as the metric for UI optimization under different interface and algorithm setups. We demonstrate the usage of this UI design method in several QA4ML applications.

CCS Concepts: • **Human-centered computing** → **HCI design and evaluation methods** • **Information systems** → *Multimedia information systems*;

Additional Key Words and Phrases: Model-based evaluation, quality assurance, interactive machine learning, data labeling, classification

ACM Reference format:

Yu Zhang, Martijn Tennekes, Tim de Jong, Lyana Curier, Bob Coecke, and Min Chen. 2024. Simulation-based Optimization of User Interfaces for Quality-assuring Machine Learning Model Predictions. *ACM Trans. Interact. Intell. Syst.* 14, 1, Article 1 (January 2024), 32 pages.

<https://doi.org/10.1145/3594552>

1 INTRODUCTION

Machine learning (ML) models rarely achieve 100% prediction accuracy in real-world applications. In some applications, such as search engines and product recommendations, the end-users

This work has been made possible by the Network of European Data Scientists (NeEDS), a Research and Innovation Staff Exchange (RISE) project under the Marie Skłodowska-Curie Program.

Authors' addresses: Y. Zhang, B. Coecke, and M. Chen, University of Oxford, Oxford, OX1 3QD, United Kingdom; emails: {yu.zhang, bob.coecke}@cs.ox.ac.uk, min.chen@eng.ox.ac.uk; M. Tennekes and T. de Jong, Statistics Netherlands, Heerlen, 6401 CZ, The Netherlands; emails: {m.tennekes, tja.dejong}@cbs.nl; L. Curier, Open University of the Netherlands, Heerlen, 6401 DL, The Netherlands; email: lyana.curier@ou.nl.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from [permissions@acm.org](https://permissions.acm.org).

© 2024 Copyright held by the owner/author(s). Publication rights licensed to ACM.

2160-6455/2024/01-ART1 \$15.00

<https://doi.org/10.1145/3594552>

can ignore prediction errors with little cost, and thus the errors are tolerable. However, prediction errors cannot be ignored in many other applications, such as medical diagnosis, fraud detection, and content moderation. Such applications are characterized as *quality-sensitive*, where rigorous **quality assurance (QA)** of model predictions is necessary.

During QA processes, human needs to review machine predictions and correct erroneous predictions in a **user interface (UI)**. This work focuses on interfaces for performing such repetitive tasks of quality-assuring ML model predictions. We refer to these interfaces as **QA for ML (QA4ML)** interfaces.

In the software development lifecycle of an interactive system, it is important to explore and evaluate many different design options to select a relatively optimized design to be implemented, tested, and deployed. The design optimization process can also be applied iteratively in the lifecycle to improve a user interface. Since most ML models are developed to handle big data, a QA4ML interface is typically used in conjunction with a huge dataset. With such an interface, users often have to spend much time performing laborious operations of viewing data objects and machine predictions and correcting errors when prediction errors are identified. For such labor-intensive QA4ML processes, it is desirable to evaluate a diverse range of interface variants to identify the design option that maximizes the user's productivity. Due to the labor-intensive nature of QA4ML, this article focuses on the evaluation metric of *task completion time* in QA4ML.

It is common to adopt a user-centered design process [12, 58] to design a user interface following a requirements analysis phase. After the software is developed, one conducts usability studies to evaluate the user interface. Such user studies can inform the improvement of the user interface at different stages of the software lifecycle [16, 59]. While opinions of potential users and usability studies can assist the design of such interfaces, these studies are usually costly and time-consuming as they intensively involve human subjects. It is difficult to evaluate a large number of design options while taking into account various factors, such as users' skills and strategies in performing a large collection of operations. It is also difficult to conduct user studies during the design phase, i.e., before a user interface has been developed.

This work proposes a model-based evaluation approach to efficiently evaluate QA4ML interfaces by modeling and simulating the task completion time. We specifically focus on grid-based interfaces for QA4ML, which are common in the literature on data labeling systems. Our approach is based on a model of the user's routine operations in QA4ML processes. Our approach can be seen as an adaptation of the Keystroke-Level Model to estimate quality assurance tasks' completion time in grid-based interfaces [15].

Using simulation, UI specialists (e.g., designers, developers, and evaluators) can explore design options at a scale unattainable with user studies. We demonstrate applying this approach to optimize QA4ML interfaces in data extraction and aerial image classification systems.

We model and simulate the influence of various factors on the QA4ML time cost. In this article, we describe the model in a manner of increasing its complexity gradually. In Section 4, we describe our simulation approach, overview the simulation model, and outline the factors to be modeled and simulated. We start with modeling simple QA4ML interfaces that only provide single edit commands. Section 5 focuses on the factors of interface layouts and application scenarios. The simple modeling exercise demonstrates the feasibility of using simulation to guide the optimization of the interface layout. The availability of interaction functions in QA4ML interfaces and algorithmic modules may affect the time cost. Batch edit functions enable users to edit the label of multiple data objects simultaneously. Section 6 investigates the influence of introducing batch edit functions. After adding the batch edit commands, users may accomplish QA4ML tasks with different combinations of single or batch edit commands. In Section 6, we introduce the factors about different user strategies in combining different commands. The accuracy of the predictions

made by the ML model is a factor that influences the overall QA4ML time cost. In Section 7, we investigate the modeling of the impact of this factor. While QA4ML interfaces may randomly present data objects to users, intelligently ordering data objects can enable more uses of batch edit commands. In Section 8, we bring into the simulation model the factor of algorithmic rank functions for ordering data objects. In Section 9, we summarize the overall simulation model with all the factors integrated.

The main contributions of this work include the following:

- a simulation *model* of users' routine operation sequence and operation time costs in quality assurance interfaces, and
- an *application* of the model to evaluate and optimize quality assurance interfaces for data extraction and aerial image classification.

2 RELATED WORK

In the following, we review label quality assurance interfaces in interactive machine learning, which are the subjects of our evaluation method. We provide a brief context of usability evaluation methods and then focus on the model-based evaluation approach to which our method belongs.

2.1 Quality Assurance in Semi-automatic Labeling

In interactive machine learning processes, the user may intervene various steps, such as feature selection [2], model tuning [17], testing [18], and explanation [53]. In quality-sensitive applications, quality assurance of model predictions and error identification is important [36]. Among the various steps the user may intervene in, a common form of user involvement is to label data for iterative model training [22]. For example, interactive machine learning systems for information retrieval may utilize relevance feedback explicitly or implicitly labeled by the user to iteratively improve the relevance of the retrieved result [32]. To reduce user burden, a common strategy is for the system to support semi-automatic labeling by learning from the user inputs and providing default labels. This way, the user only needs to quality-assure the default labels and correct mistakes, which typically takes less time than labeling from scratch.

Typically, the default labels to be quality-assured in semi-automatic labeling are provided by machine-learned models. Fluid Annotation [6] uses a pre-trained neural network to propose a set of possibly overlapping segments to help annotate image segmentation. The user can then edit the geometry of the segments to correct errors. V-Awake [27] focuses on time series segmentation. It uses LSTM to assign tentative labels and visualizes the model information to help annotators diagnose and correct model predictions.

In some cases, the default labels to be quality-assured may also come from crowdworkers. LabelInspect [44] is a visual analytics system for improving crowdsourced image labels. It enables the user to identify uncertain instance labels and unreliable crowdworkers. DataDebugger [60] is a system for image label correction based on data debugging using trusted items. It allows the user to identify and correct suspicious data labels, specify data labels to be trusted, and propagate the trusted labels.

When the default labels come from machine predictions, the machine assistance may be gradually improved by incrementally learning from user inputs to reduce users' quality assurance effort. Fails and Olsen propose Crayons [23] for image pixel classification. The user can paint pixels to correct machine predictions and simultaneously provide training data to the machine. Fogarty et al. develop a search system, CueFlik [25], with which the user can define search criteria for a concept by providing positive and negative examples and rank search results according to the similarity to the concept. ISSE [14] algorithmically suggests refined segmentation to help annotators separate sound into its respective sources by painting on time-frequency visualizations.

2.2 Usability Evaluation

Evaluation is helpful in various stages of iterative software engineering processes for interactive systems. During prototyping, evaluation may guide software development to improve usability. After implementation, evaluation may derive summative results on whether the interactive system achieves specific goals, such as improved efficiency over existing systems.

2.2.1 Overview. Usability evaluation methods can be categorized by two dimensions: involvement of users and methodological approach [9]. An empirical evaluation involves a sample of target users (also referred to as “testing method” [5]). By comparison, an analytical evaluation usually requires evaluation experts to carry out (also referred to as “inspection method” [5] and “expert analysis” [21]). The evaluation methodology can be qualitative, quantitative, or combined.

Empirical evaluation captures payoff measures, such as user performance, to infer interface properties [29] with the following example methods:

- *Laboratory studies* involve a sample of users to perform tasks and analyze objective measurements, such as task completion time and accuracy. It is prevalent in HCI research nowadays.
- *Think-aloud* gains insight about the interface through the records of what the user said that came into their mind while performing tasks with the interface.
- *Questionnaires*, such as Likert scales, gather user ratings of interfaces.

Analytical evaluation inspects the interface’s intrinsic properties influencing usability [29] with the following methods being examples.

- *Heuristic evaluation* involves a team of evaluators checking an interface against a short list of design guidelines and aggregating the identified issues [46].
- *Cognitive walkthroughs* are a technique where evaluators mentally walk through a task following a novice user’s mindset to identify usability issues [42].
- *Model-based evaluation* involves modeling users’ physical and mental operations to accomplish a task in an interface [50]. It is also referred to as “action analysis” [43]. The modeling enables the estimation of usability metrics, such as task completion time [15] and learning time costs [37].

Analytical methods are usually carried out by evaluation experts. Analytical methods do not directly measure the performance of target users and thus may not be as conclusive as empirical methods. Meanwhile, analytical methods can be less costly than empirical methods, as the former typically involve fewer people and thus are more suited for early-stage design and prototyping iterations. Additionally, it is usually easier to attribute the causes of usability issues with analytical methods. Our approach belongs to analytical methods, specifically model-based evaluation.

2.2.2 Model-based Evaluation. We use the model-based evaluation approach to evaluate quality assurance interfaces in interactive machine learning. Generally, model-based evaluation involves modeling the user’s task completion procedure in an interactive system to estimate usability metrics, such as task completion time or learning time. Kieras identifies three categories of models in model-based evaluation: task network models, cognitive architecture models, and GOMS models [50]. We introduce examples following this categorization.

- *Task network models* capture the dependencies between user and machine tasks as networks [41]. **Systems Analysis of Integrated Networks of Tasks (SAINT)** is a simulation language developed for task network models [19]. Given the distributions of

probabilistic variables in the model, it supports the estimation of task completion time distribution through Monte Carlo experiments.

- *Cognitive architecture models* apply theories in cognitive science to model human motor control, perception, and cognition [3]. Examples include the ACT-R architecture [4] and the EPIC architecture [40]. Both of them model users' low-level interaction with production rules that depict user reactions to stimuli but differ in some aspects of their human model. ACT-R concerns serial production cycles, while EPIC concerns parallel production cycles.
- *GOMS models* describe the required procedural knowledge for a user to carry out routine tasks in user interfaces, similar to depicting computer algorithms with pseudocode [33, 34]. The term "GOMS" stands for its four components: goals, operators, methods, and selection rules. Various GOMS techniques exist, such as the Keystroke-Level Model [15]. Our work falls into this category.

Model-based evaluation has been used in various application scenarios. The EPIC architecture has been used to predict human performance in telephone operator tasks [40]. John and Vera use GOMS to model the highly interactive task of video game playing [35]. Gond and Kieras use GOMS to identify usability issues and aid the redesign of ergonomics assessment software [28]. Beard et al. apply GOMS to evaluate the user interface for displaying computed tomography images for medical diagnosis [11]. Kieras and Santoro apply GOMS to evaluate the user interface and collaboration scheme for shipboard workstations [39]. Kieras and Hornof introduce a GOMS model for the visual search task [38]. Ramkumar et al. apply GOMS analysis to image segmentation interfaces [48]. Azzopardi et al. model and simulate the tradeoff between the query and assess interactions to the total time cost in information retrieval systems [7] and validate the simulation result through a user study [8]. Like this line of work, our modeling and simulation follow the model-based evaluation approach. We focus on a different scenario of evaluating data extraction and aerial image classification systems.

Task completion time is a common metric in usability evaluation. To estimate task completion time, the model-based evaluation typically involves the modeling or reusing existing models of user operations' time costs. For example, Fitts' law [24] has long been used to model the time costs of users' object-pointing operations [45]. It has been extended to the time cost modeling in related mouse movement tasks, such as trajectory-based tasks [1] and object-pointing tasks with multi-scale navigation [30].

With models in model-based evaluation, redesign and optimization of user interfaces can be conducted computationally and swiftly, as demonstrated in the computational interaction literature [47]. Gajos and Weld propose the SUPPLE system for automatic user interface generation through constrained optimization of users' navigation costs [26]. The optimization is based on the interface model with functional specifications, the model of the rendering device, and the users' usage trace in the interface. Zhang and Zhai propose a card-playing model for information retrieval interfaces and demonstrate adaptive optimization of the interface according to the screen size and user's interaction sequence [64]. Todi et al. propose an adaptive user interface technique that uses Fitts' law to estimate user interaction cost and reinforcement learning to plan sequences of adaptations [57].

Like this line of research, we demonstrate using model-based evaluation to optimize interfaces, with estimated task completion time being the metric. Meanwhile, our model is dedicated to the scenario of quality assurance in interactive machine learning. While evaluating interactive machine learning systems is generally challenging [13], our work suggests that some of the most laborious routine processes in interactive machine learning, such as quality assurance, can be modeled effectively and evaluated efficiently.

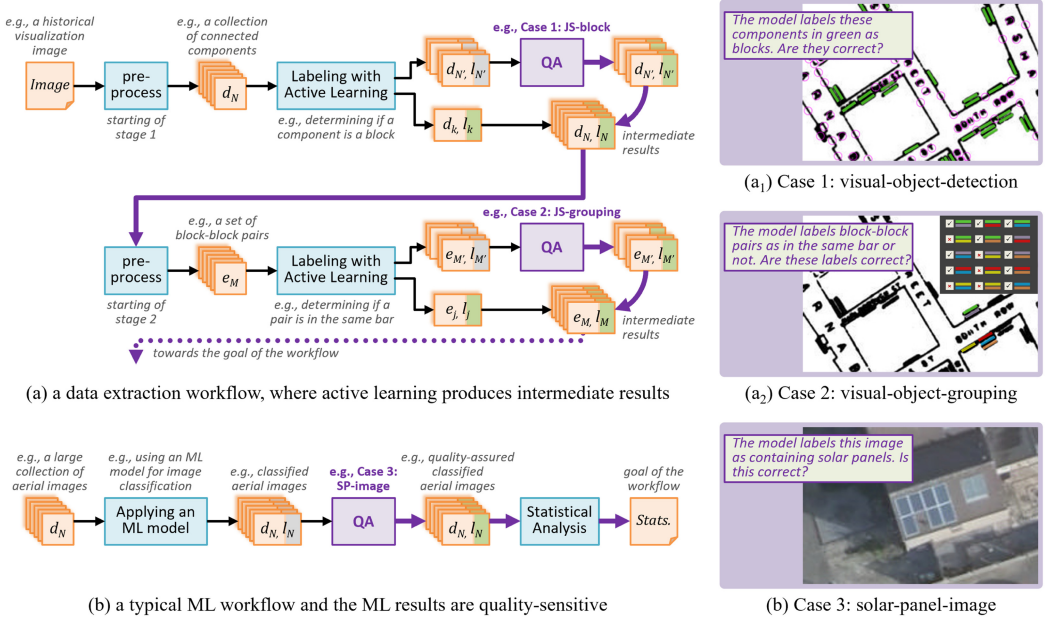


Fig. 1. ML workflows in data-centric applications requiring quality assurance: (a) A workflow with two stages (a₁ and a₂) to extract datasets from visualization images. Quality assurance is necessary to avoid propagating errors to the subsequent stage and guarantee the correctness of the extracted dataset. (b) A workflow to generate summary statistics from the classification of whether an image contains solar panels. Quality assurance of classification labels is needed to achieve reliable statistics.

A limitation of model-based evaluation methods is their restriction to routine and well-defined user tasks [33]. For exploratory tasks where the goal is ill defined and requires users' complex reasoning procedures and creativity, it is intractable for the evaluators to model the usage procedures. This problem is absent in the case of quality assurance that we focus on, which has well-defined goals and relatively predictable usage procedures. Thus, we can derive a model of the user operation sequences.

3 BACKGROUND

This section introduces two example data-centric applications and their QA4ML interfaces to illustrate the need for QA4ML. Then, we present a template grid-based interface that typifies the QA4ML interfaces.

3.1 Data-centric Applications

Visualization Data Extraction: The first application concerns detecting and grouping visual objects in historical visualizations as introduced by Zhang et al. [62]. For example, given John Snow's famous cholera map [52], the rectangular blocks are to be detected (Figure 1(a₁)) and grouped (Figure 1(a₂)). The detected visual objects are then used to extract the underlying datasets generating the visualizations. While machine-learned models may not accurately detect the visual objects, they can suggest visual objects and tentative classification labels for the user to quality-assure, as demonstrated in the MI3 workflows [62]. In this application, QA is needed for the correctness of the extracted datasets.

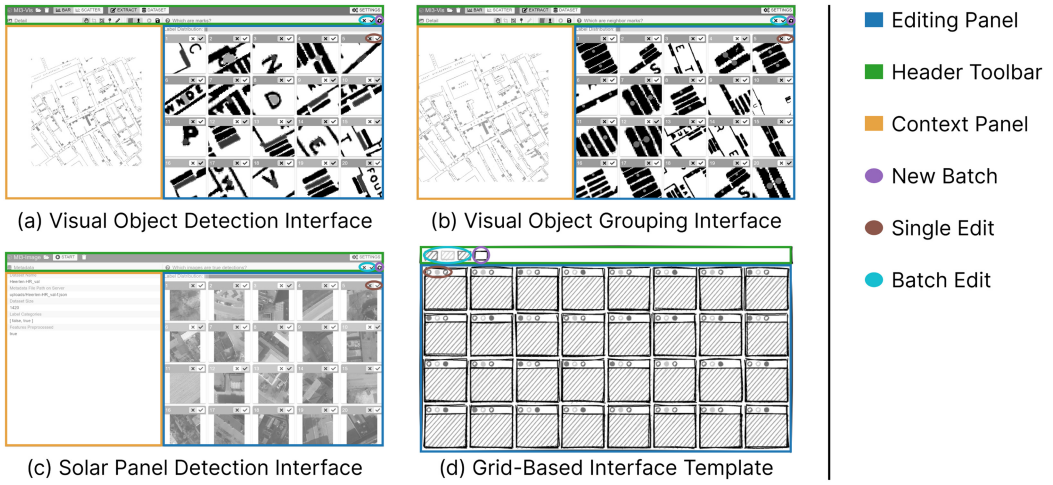


Fig. 2. Examples of QA4ML interfaces: The interface components (editing panel, header toolbar, and context panel) are annotated with colored rectangles. The buttons corresponding to operations (new batch, single edit, batch edit) available to the user are annotated with colored ovals. The interfaces all contain an edit panel that presents a matrix of thumbnails showing data objects, a header toolbar providing interaction functions, and a context panel that provides an overview of the dataset: (a) An interface for visual object detection corresponding to workflow in Figure 1(a₁). (b) An interface for visual object grouping corresponding to workflow in Figure 1(a₂). (c) An interface for solar panel detection corresponding to workflow in Figure 1(b). (d) The typical design of grid-based interfaces in QA4ML.

Solar Panel Detection: The second application concerns generating a labeled benchmark image dataset where labels classify aerial images as containing solar panels or not (Figure 1(b)). The benchmark dataset is used to generate official statistics on solar panel deployment. In this application, while using ML models' predictions may accelerate the labeling, predictions not checked manually are imprecise and untrustworthy. A QA process inspecting and correcting machine predictions is necessary for the correctness of the official statistics. Figure 1(b) shows this process of applying an ML model and quality-assuring machine predictions to generate a benchmark dataset and, consequently, official statistics.

These applications typify scenarios of carrying out real-world *data-centric* ML projects. In these scenarios, the ultimate goal is to gather high-quality labeled/processed data. Using ML in these projects is not to train accurate models but to assist the data labeling and processing. These applications exhibit a common challenge: A user may need to perform many tedious quality assurance operations to correct machine prediction errors. It is critical to optimize QA4ML interfaces to reduce user operation costs.

3.2 Grid-based Interfaces

QA4ML requires users to verify machine predictions in interfaces. Figure 2 shows three QA4ML interfaces corresponding to the two applications described above. Figure 2(a) and (b) show interfaces for classification in visualization data extraction. The interface in Figure 2(a) displays a batch of visual objects. The visual objects are to be categorized as encoding data or not. The context panel on the left shows part of John Snow's cholera map under check. Figure 2(c) shows an interface for classifying solar panel detection. In this interface, an ML model assigns tentative classification labels for the images on whether they contain solar panels. In these interfaces, ML models are used to extract and group visual objects.

The quality assurance interface can be divided into three parts: the editing panel, header toolbar, and context panel. The editing panel is where a user performs most operations to correct ML prediction errors. Because data objects to be inspected are usually too many to be quality-assured all at once, the editing panel normally shows a subset (referred to as a “batch”) of data objects each time. The header toolbar may provide utilities such as dataset upload, requesting the system to present the next batch of data objects to be quality assured, and simultaneously setting labels for a batch of data objects. For example, in Figure 2(a)–(c), each interface shows a batch of 20 data objects. The context panel gives the user a dataset overview and may present summary statistics about the labeling progress, such as the machine errors identified. The following mainly focuses on the editing panel and header toolbar as they accommodate the functions directly related to quality assurance.

In all the interfaces, users need to manually verify machine predictions in a grid panel on the right part of the interface. The grid panel presents each data object as a thumbnail image in a grid cell. Each grid cell provides buttons for editing its label. Figure 2(d) typifies the design of grid-based interfaces. This article focuses on grid-based interfaces in QA4ML. We refer to each thumbnail corresponding to a data object and an interaction mechanism (e.g., buttons for editing label categories) as a *grid cell*, and all grid cells in the editing panel as a *batch*. It is the most commonly adopted design in QA4ML interfaces according to Zhang et al.’s survey [63] with numerous instances in the literature [10, 31, 44, 60, 62].

We list typical functionalities of QA4ML interfaces exhibited in the literature. The interface enables a user to *request a new batch* of data objects to be quality assured in the editing panel. An algorithmic process (e.g., active learning or clustering) may determine the priority for data objects to be presented to the user. An ML model assigns default labels to the data objects. The user has to inspect data objects to judge whether the default labels are correct. The interface provides *single edit* commands in the editing panel for correcting individual label errors. To improve efficiency, the interface provides the user with *batch edit* commands for changing the labels of a group of data objects (e.g., set all labels to positive).

QA4ML interfaces utilize human intelligence to

- recognize ML model’s prediction errors, and
- determine whether using a batch edit command is more efficient than applying single edit commands to data objects individually.

Batch editing is beneficial when the batch of data objects mostly shares the same label. Consider a batch of 20 data objects with binary labels (e.g., “yes” or “no”). Suppose there are 10 true-positive labels, 2 false-positive labels, and 8 false-negative labels. In this case, the user can use a batch edit command to set all labels to positives first and then change the 2 false-positive labels to negatives. This process requires 3 user operations. If the user only uses single edit commands, then 10 operations are needed.

QA4ML interfaces also utilize machine assistance to

- assign default labels to data objects, and
- determine the ordering for data objects to be labeled.

Suitable ordering may accelerate the quality assurance process. First, the ordering algorithm may bunch together data objects of the same label, providing more opportunities for applying a batch edit command. Second, when the ML model for assigning default labels is incrementally updated with the user corrections, ordering generated by active learning methods may enable the ML model to learn more efficiently, leading to better default labels.

While human-in-the-loop is indispensable for QA4ML processes, it is also essential to evaluate and optimize QA4ML interfaces to reduce user effort. Meanwhile, evaluating different design

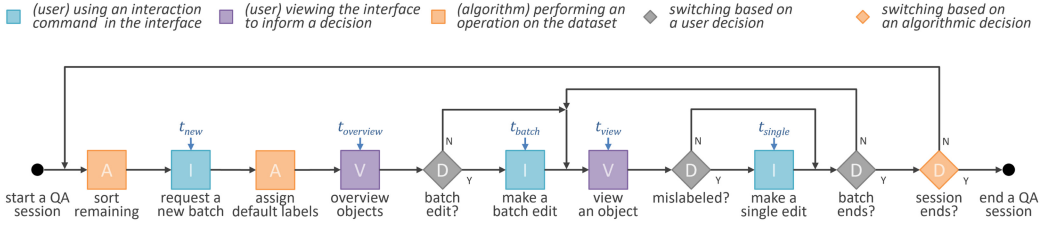


Fig. 3. A general workflow of user and machine operations in a quality assurance session in a grid-based interface typified in Figure 2(d). We group the operations into five categories: user interaction in the interface, the user’s viewing action, algorithmic operation, user decision, and algorithmic decision.

options of QA4ML interfaces with user-centered evaluation methods (e.g., surveys, group discussions, controlled experiments) is usually time-consuming for designers and potential users participating in the evaluation.

4 OVERVIEW OF THE SIMULATION APPROACH

This section outlines a model-based approach to evaluating quality assurance interfaces. Section 4.1 introduces a general workflow capturing the user’s routine operations in grid-based interfaces for QA4ML. We describe modules in the workflow to be modeled and simulated. The workflow is constructed by analyzing user and machine operations in the QA4ML interfaces typified in Figure 2(d). Given the workflow, Section 4.2 introduces a simulation model. The model estimates the time cost of a QA4ML session by simulating sequences of user and machine operations and adding up the time cost of each operation. Section 4.3 outlines the factors we model and simulate. Section 4.4 clarifies the scope and assumptions of the simulation approach.

4.1 A Routine Task Workflow

4.1.1 Problem Formulation. In general, quality assurance concerns the following setup. A dataset $X = \{x_i | i = 1, 2, \dots, n\}$ contains n data objects to be assigned quality-assured labels. An ML model is applied to the dataset X , generating default labels $Y = \{y_i | i = 1, 2, \dots, n\}$, where y_i is the default label for x_i . The user needs to quality-assure the default labels to generate verified labels $Y^* = \{y_i^* | i = 1, 2, \dots, n\}$.

We refer to the entire process of carrying out operations to quality-assure the n data objects as a *session*. In typical QA4ML interfaces, a subset of data objects, $X_i \subseteq X$, is loaded into the editing panel. The editing panel can accommodate maximal n_{batch} data objects, and a subset of data objects loaded to the editing panel is of size K ($K \leq n_{batch}$). We refer to the process of quality-assuring the subset X_i as a *round*.

For classification tasks, let c be the number of label categories. Let $cm \in [0, 1]_{c \times c}$ be the normalized confusion matrix with $\sum_{i,j} cm_{i,j} = 1$. $cm_{i,j}$ denotes the rate of the data objects with true label i and default label j over all the n data objects.

For binary classification, the correctness of each label y_i can be represented with one of the four values: **true-positive (TP)**, **true-negative (TN)**, **false-positive (FP)**, and **false-negative (FN)**. Let n_{TP} , n_{TN} , n_{FP} , n_{FN} be the number of TPs, TNs, FPs, and FNs in the whole dataset. The normalized confusion matrix can be represented as $cm = \frac{1}{n} \begin{pmatrix} n_{TN} & n_{FP} \\ n_{FN} & n_{TP} \end{pmatrix}$.

4.1.2 Operators in the Workflow. Section 3.2 introduces user operations and machine operations in typical grid-based interfaces for QA4ML. Using the user and machine operations as building blocks, we summarize the quality assurance workflow as shown in Figure 3. The workflow includes two nested loops. The inner loop is between the operations “view an object” and “batch

ends?” Going through the inner loop once quality-assures the label of a data object. The outer loop is between the operations “sorting remaining” and “session ends?” Going through the outer loop once quality-assures the labels of a batch of data objects. In the following, we introduce each operation in the workflow.

The workflow involves a set of interaction commands, or operators, initiated by the user (shown as blue rectangles in Figure 3), including the following:

- *Request a new batch* of data objects to quality-assure by activating a “new batch” command in the interface (denote its average time cost as t_{new}).
- *Make a batch edit* to assign the same label to the current batch of data objects by activating a “batch edit” command in the interface (time cost t_{batch}).
- *Make a single edit* to edit the label of a data object in the current batch by activating a “single edit” command in the interface (time cost t_{single}).

Some of the interaction commands can only be issued after the user makes explicit decisions (shown as gray diamonds in Figure 3), including the following:

- *Batch Edit?* The decision of whether a “batch edit” should be issued, which depends on whether using a batch edit saves effort over only using single edits.
- *Mislabeled?* The decision of whether a “single edit” should be issued, which depends on whether the data object is mislabeled.
- *Batch Ends?* The decision of whether all the data objects in the current batch are quality-assured.

The user needs to perform viewing actions (shown as purple rectangles in Figure 3) to obtain the information of the interface state to inform some of the decisions, including the following:

- *Overview objects* in the current batch to comprehend the overall label category distribution (time cost $t_{overview}$).
- *View an object* in the current batch to comprehend its label category and whether it is mislabeled (time cost t_{view}).

The machine needs to perform algorithmic operations, including the following:

- *Sort remaining* to reorder remaining data objects to be quality assured. In a QA4ML interface, the sorting can be implemented with any algorithm that serves the reordering purpose. For example, the rank method can be implemented with a clustering algorithm to group data objects of the same label category [20, 55, 56]. In this case, the rank method may create good opportunities to use batch edit commands. The sorting may also be implemented by scoring functions in active learning [51] to prioritize data objects whose labels are more uncertain.
- *Assign default labels* to the batch of selected data objects. In the scenario of quality-assuring ML predictions, default labels may be assigned by a pre-trained machine learning model or an incrementally updated model [6, 14, 27]. In general quality assurance scenarios, default labels may come from other sources, such as another user or pre-defined rules.

An algorithmic decision (*session ends?*) determines whether the session should end, which depends on whether all the data objects are quality-assured (shown as an orange diamond in Figure 3).

The original Keystroke-Level Model decomposes the interaction tasks into the keystroke-level operators, such as keystrokes, pointing, and mental preparation for physical actions, and assumes each type of operator costs the same amount of time. To estimate the task completion time more

accurately, we decide not to make this assumption. For example, we do not assume the time cost of “batch edit” (t_{batch}) and “single edit” (t_{single}) to be the same, considering that the buttons for batch edit and single edit are distributed at different places in the interface. By comparison, in the Keystroke-Level Model, “batch edit” and “single edit” are both a pointing followed by a clicking, and thus should be assumed to cost the same amount of time.

4.2 A Simulation Model

The workflow provides the basis of a simulation model and defines the top-level structure of a simulator implementing the model. Algorithm 1 illustrates an implementation of a simulator for QA4ML sessions. The simulation algorithm generates a mock dataset, synthesizes a sequence of operations to quality-assure the mock dataset, and adds up the operation time costs.

ALGORITHM 1: QA4ML Session Simulation

Inputs: $n, n_{batch}, \vec{t}$

Output: time cost of the quality assurance session $T_{session}$

```

1:  $X, Y^* \leftarrow \text{CreateMockDataset}(n)$ 
2:  $ops \leftarrow []$  ▷ log of the operation sequence
3: do
4:    $X, Y \leftarrow \text{Rank}(X, Y)$  ▷ sorting remaining
5:    $ops.push(\text{"new"})$  ▷ request a new batch
6:    $X_s \leftarrow X.slice(n_{batch})$ 
7:    $Y_s^* \leftarrow Y^*.slice(n_{batch})$ 
8:    $Y_s \leftarrow \text{AssignDefaultLabels}(X_s)$  ▷ assign default labels
9:    $ops.push(\text{"overview"})$  ▷ overview objects
10:   $cmd \leftarrow \text{SelectEditCommand}(Y_s, Y_s^*)$ 
11:  if  $cmd \in \text{BatchEditCommands}$  then ▷ batch edit?
12:     $ops.push(cmd)$  ▷ make a batch edit
13:     $Y_s \leftarrow \text{ApplyBatchEdit}(cmd, Y_s)$ 
14:  end if
15:  for  $j \in (1, \dots, Y_s.length)$  do ▷ loop until batch ends
16:     $ops.push(\text{"view"})$  ▷ view an object
17:    if  $Y_s[j] \neq Y_s^*[j]$  then ▷ mislabeled?
18:       $ops.push(\text{"single"})$  ▷ make a single edit
19:    end if
20:  end for
21: while  $Y \neq []$  ▷ session ends?
22:   $T_{session} \leftarrow \text{GetCost}(ops, \vec{t})$  ▷ compute total time cost
23: return  $T_{session}$ 

```

The simulation algorithm takes as input the number of data objects n , the number of data objects presented simultaneously in the editing panel n_{batch} , and time costs of the operations in the workflow $\vec{t}$. It outputs the total time cost of the QA session.

Algorithm 1 is generic in that different implementations of routines such as “CreateMockDataset,” “Rank,” and “AssignDefaultLabels” can be plugged in to simulate different QA4ML interface designs. For example, at the beginning of the simulation (line 1 of Algorithm 1), a mock dataset is generated by “CreateMockDataset.” This routine may fetch real data objects as is in the simulations in Section 5. It may alternatively generate a mock dataset according to n and an additional parameter of the default labels’ confusion matrix, as is in the simulations in Sections 6, 7, and 8.

4.3 Modeled and Simulated Factors

This section outlines factors of QA4ML interfaces that may affect the total time cost of the quality assurance process. These factors inform our modeling and simulations.

- **Interface layout:** The grid-based interface may utilize different layouts (as shown in Figure 6). The display size of each data object is affected by the layout, assuming the screen size is fixed. The display size may affect the time cost to comprehend the data object's label (t_{view} and $t_{overview}$). Additionally, the layout determines the number of data objects displayed simultaneously (n_{batch}), affecting the effectiveness of batch edit commands. For layouts with larger n_{batch} , issuing a batch edit command can edit the labels of more data objects. Section 5 discusses this factor.
- **Application scenario:** The user operation time costs may depend on the quality assurance process's application scenarios (e.g., the ones introduced in Section 3.1). For example, the complexity and time cost of comprehending the data labels (t_{view} and $t_{overview}$) may depend on the application scenario. Section 5 discusses this factor. We model user operation time costs as functions of the *layout* of the grid-based interface and the *application* from which the dataset to be quality-assured comes.
- **Availability of interface functions:** Section 4.1 introduces a general QA4ML workflow that concerns several functions commonly but not always offered in QA4ML interfaces, such as batch editing and the rank method. The availability of these functions may affect the time cost of the QA4ML process. Section 6 discusses the influence of introducing batch edit commands. Section 8 discusses the influence of introducing rank methods.
- **User's label strategy:** As the QA4ML interface provides more functionalities, there may be multiple methods to accomplish the same goal. The user's strategy to select a method for accomplishing the goal affects the time cost. Section 6 discusses this factor.
- **Default label accuracy:** The accuracy of default labels affects the number of times the user needs to use single edit and batch edit commands. Section 7 discusses this factor.
- **Rank method:** The rank method may change the distribution of label categories in each batch. Effective rank methods may alter the distribution to create more opportunities for batch edit commands to label multiple data objects with one click. Section 8 discusses this factor.

4.4 Scope and Assumptions

In the following sections, we set the following restrictions on the scope of our simulation-based evaluation:

- We focus on the scenario of *quality assurance of classification labels*. The following sections analyze the quality assurance of binary classification labels, with the two label categories being "positive" and "negative." Our analysis can easily be extrapolated to multi-class classification.
- We focus on the *task completion time* as the evaluation metric.

We make the following assumptions to simplify the modeling and simulations:

- We assume *error-free execution* in the user operations. The user is assumed to be able to recognize the correct labels of the data objects and does not accidentally activate incorrect commands when using QA4ML interfaces. Fully capturing the user strategies and potential mistakes is intractable due to the current insufficient understanding of the human decision process. Thus, a general strategy in model-based evaluation methods is to model what the users should do instead of what the users will do [50]. We essentially give a lower-bound

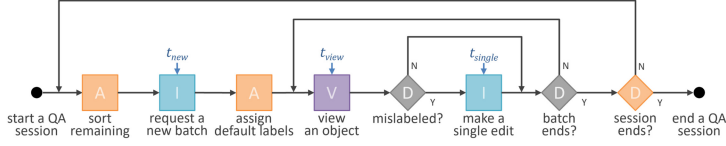


Fig. 4. A workflow of human and machine operations in a grid-based interface without batch edit functions. This workflow simplifies the workflow in Figure 3 by removing operations related to batch edit.

estimation by adopting the original assumption in the Keystroke-Level Model that the user makes no mistakes.

- We assume the *machine operations are fast* and their time costs are ignorable compared with user operation time costs. Thus, we focus on the time costs of user operations.
- We assume the *user operations are sequential* instead of parallel. Under this assumption, $GetCost$ in Algorithm 1 is a simple summation. $GetCost(ops, \vec{t}) = \sum_{op} N_{op} t_{op}$ where op denotes a user operation type, N_{op} denotes the number of operations of type op , and t_{op} denotes the time cost to execute the operation of type op once.

The following sections introduce different technical aspects of our simulation model. We gradually bring out factors to be modeled, increasing the model's complexity.

5 THE FACTORS OF INTERFACE LAYOUT AND APPLICATION

The following introduces simulations of task completion time with different interface layouts and application scenarios. We introduce a method to estimate user operations' time costs in the grid panel for editing labels. We show how simulation can help find an optimal layout for the grid-based interface. This section considers a simplified QA4ML interface template compared to Section 3.2. This simplified interface template provides the new batch and single edit commands but does not provide batch edit commands. Figure 4 shows its workflow.

5.1 Datasets to be Quality-assured

In the simulation, we consider three real datasets to be quality-assured from the QA4ML application scenarios introduced in Section 3.1.

Visual Object Detection: This scenario concerns detecting visual objects from historical visualizations as introduced in Zhang et al.'s work [62] (see Figure 1(a₁)). An ML model has assigned default labels to 3,975 candidates of visual objects in John Snow's cholera map [52]. A post hoc analysis reveals that the model predictions have a normalized confusion matrix $cm = \begin{pmatrix} 86.98\% & 0.87\% \\ 0.53\% & 11.61\% \end{pmatrix}$, i.e., $\frac{n_{TN}}{n} = 86.98\%$, $\frac{n_{FP}}{n} = 0.87\%$, $\frac{n_{FN}}{n} = 0.53\%$, and $\frac{n_{TP}}{n} = 11.61\%$. We refer to this dataset as *JS-block*.

Visual Object Grouping: This scenario concerns classifying pairs of visual objects to group visual objects as introduced in Zhang et al.'s work [62] (see Figure 1(a₂)). An ML model has assigned default labels to 782 candidates of visual object pairs to determine whether they should be grouped. A post hoc analysis reveals that the model predictions have a normalized confusion matrix $cm = \begin{pmatrix} 70.15\% & 0.12\% \\ 0.36\% & 29.37\% \end{pmatrix}$. We refer to this dataset as *JS-grouping*.

Solar Panel Detection: This scenario concerns classifying aerial images to detect solar panels (see Figure 1(b)). This dataset is provided by Statistics Netherlands.¹ An ML model assigned

¹<https://www.cbs.nl/>.

default labels to 1,278 aerial images. A post hoc analysis reveals that the model predictions have a normalized confusion matrix $cm = \begin{pmatrix} 64.82\% & 5.60\% \\ 5.44\% & 24.14\% \end{pmatrix}$. We refer to this dataset as *SP-image*.

5.2 Conceptual Model for Simulation

Figure 4 shows a QA4ML workflow. Consider a grid panel with K grid cells. K_{FP} and K_{FN} are the numbers of false-positive and false-negative labels shown in the grid panel. The grid panel provides two commands for the user: *request a new batch* and *make a single edit*. Assuming that the user operations are sequential, the total time cost of a round of quality assurance for the K grid cells is a summation of the user operation time costs:

$$T_{round} = t_{new} + K \cdot t_{view} + (K_{FP} + K_{FN}) \cdot t_{single}. \quad (1)$$

For a session with n data objects, denote its overall confusion matrix as $\begin{pmatrix} n_{TN} & n_{FP} \\ n_{FN} & n_{TP} \end{pmatrix}$. One can extrapolate T_{round} and deduce the total time cost of a session: $T_{session} = \lceil \frac{n}{K} \rceil \cdot t_{new} + n \cdot t_{view} + (n_{FP} + n_{FN}) \cdot t_{single}$. As long as we know the estimated values of operator time costs t_{new} , t_{view} , and t_{single} , we can estimate T_{round} and $T_{session}$.

5.3 Estimating Operator Time Costs

There are various approaches to estimating operator time costs t_{new} , t_{view} , and t_{single} . One may recruit potential users to use a QA4ML interface and use eye-tracking, video recording, or interaction logs to capture detailed timing data. However, such processes usually require a relatively complex experiment setup unavailable or inconvenient to most software development projects. Another common practice in the model-based evaluation literature is to reuse time costs estimated by previous work [15]. To accomplish a swift evaluation, one may also plug in hypothetical values for the operator time costs, such as assuming all types of operations cost the same. While these approaches are all feasible for the QA4ML simulation, in the following, we introduce an approach to get a realistic estimation of operator time costs swiftly.

5.3.1 Method. We conduct self-experimentation under different conditions for n_{batch} , $K_{FP} + K_{FN}$, *layout*, and *application* to gather observations of T_{round} values. Then, we use numerical analysis and modeling, such as multiple linear regression, to obtain t_{new} , t_{view} , and t_{single} values. The UI specialists can use the obtained models to inform conditions (e.g., layout designs) that have not been experimented with. Below, we outline the process to model t_{new} , t_{view} , and t_{single} . More details can be found in Appendix A in the supplementary materials.

5.3.2 Procedure and Results. The values of operator time costs t_{new} , t_{view} , and t_{single} may depend on the *layout* of the QA4ML interface and the *application*. For example, the time to view each grid cell, i.e., t_{view} , may vary for different sizes of grid cells in the editing panel (dependent on layout), and the difficulty of recognizing the data objects shown in these grid cells (dependent on application). Therefore, we estimate operator time costs separately for each layout and application.

For each application scenario introduced in Section 5.1, we create variations of QA4ML interfaces with different layouts. Figure 6 shows a few layout options of the QA4ML interface for the JS-block dataset. We have experimented with eight grid layouts for JS-block (1×1 , 1×2 , 2×2 , 2×3 , 3×4 , 4×5 , 6×6 , and 8×8), six layouts for JS-grouping (1×1 , 2×3 , 3×4 , 4×5 , 6×6 , and 8×8), and five layouts for SP-image (1×1 , 2×3 , 3×4 , 4×5 , and 5×8). For each grid layout, we take a minimum of seven samples of T_{round} with different values for K and $K_{FP} + K_{FN}$ (see Tables 1 and 2 in Appendix A).

We assume T_{round} follows Equation (1). Thus, for each combination of layout and application, given the observations with different K and $K_{FP} + K_{FN}$, we use multiple linear regression to

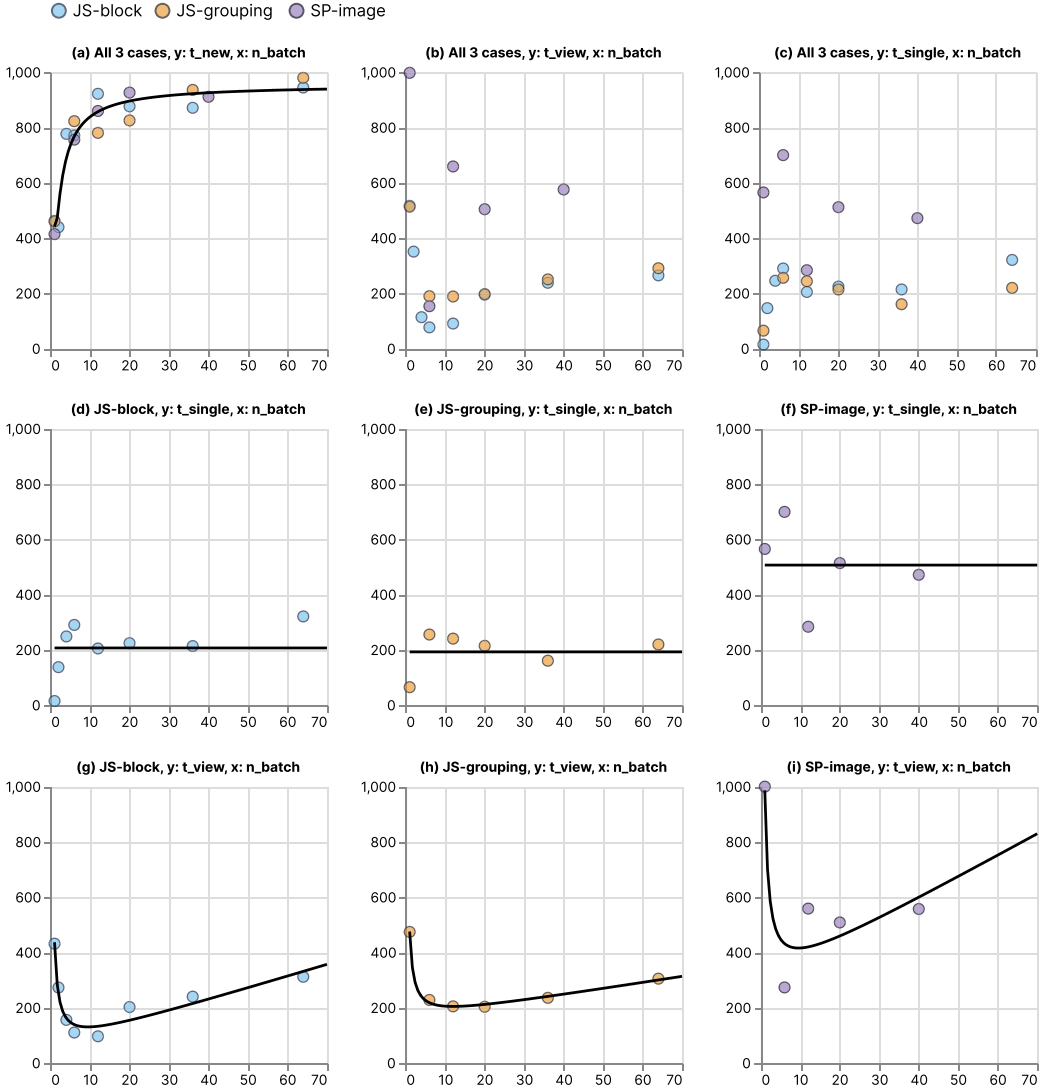


Fig. 5. Summary of the experiment result and fitted models: (a) Estimations of t_{new} by multiple linear regression. The selected model of t_{new} that we plot is $y = a + b/x + c/x^2$. (b) Initial estimations of t_{view} by multiple linear regression. (c) Initial estimations of t_{single} by multiple linear regression. ((d)–(f)) Models of t_{single} for the three applications. The selected model is $y = a$. The data points are t_{single} reestimated by putting t_{new} back. ((g)–(i)) Models of t_{view} for the three applications. The selected model is $y = a + bx + c/x$. The data points are t_{view} reestimated by putting t_{new} and t_{single} back. The x-axes of the subfigures are the number of grid cells n_{batch} . The y-axes of the subfigures are the corresponding operator time costs t_{new} , t_{view} , or t_{single} . The unit of all the time costs is milliseconds.

estimate t_{new} , t_{view} , and t_{single} values. The estimation results are shown in Figure 5(a)–(c) and numerically in Table 3 in Appendix B. The estimations of the user operation time costs in Table 3 in Appendix B are mostly statistically significant with high R^2 , which suggests that our linear model in Equation (1) is numerically accurate. Thus, our assumption that the user operations are sequential instead of parallel is suitable for modeling.

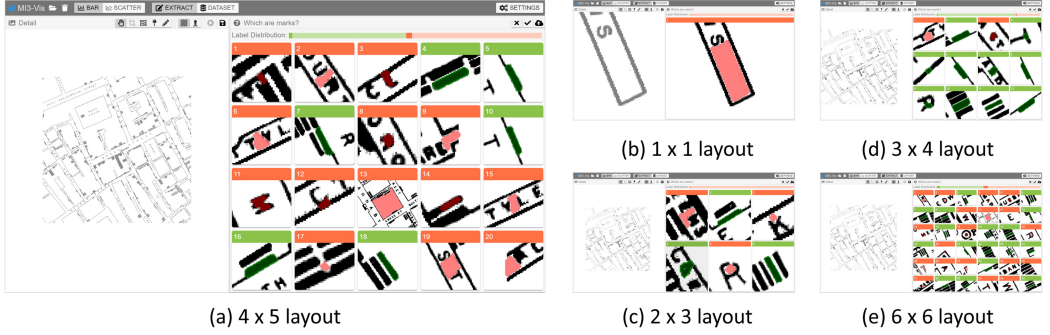


Fig. 6. Different layouts of a grid-based interface for visual object detection in the JS-block dataset: (a) 4×5 layout. (b) 1×1 layout. (c) 2×3 layout. (d) 3×4 layout. (e) 6×6 layout.

Then, for each application, we fit a function of the operator time costs regarding layout (parameterized as n_{batch}). Our purpose in fitting the function is not to obtain an ultimate model of the time costs. Using the fitted functions, we can obtain smoothed estimations of the operator time costs and predict the operator time costs for the layouts we have not experimented with.

The t_{new} values for the three application scenarios are shown in Figure 5(a) with three colors. They show similar trends concerning the number of grid cells. Based on this observation and the rationale that the time required to issue a new batch command is likely similar among the three application scenarios, we have decided to fit a single model to the three sets of data points.

5.3.3 Details on Curve Fitting. First, we use a set of elementary functions (e.g., constant, polynomial, exponential, logarithm) to fit the operator time costs as a function of n_{batch} . Then, we make attempts to formulate a semantic explanation of those functions with low error measures and abandon the functions for which we cannot find an intuitive explanation. We choose a function as the final model based on fitting errors and interpretability. Fitting the functions enables us to smooth the estimations of the operator time costs. In the following, we introduce the process of curve fitting. More details can be found in Appendix A and Appendix B.

We use n_{batch} to parameterize the layout. One may expect that for different layouts (e.g., 2×3 and 3×2) with the same n_{batch} , the operator time costs may still be different. Thus, it is possible to fit functions with two parameters, the number of rows and columns. We use only one parameter, n_{batch} , because the ratio of the number of rows and number of columns in all the layouts we have experimented with are close and are all between 1:1 and 1:2.

Fitting t_{new} : For the initial estimations of t_{new} in Figure 5(a), formulae $a + b/x$, $a + b/x + c/x^2$, and $a + b/x + c/x^2 + d/x^3$ are found by numerical analysis to be among the best candidate models for two-, three-, and four-parameter models, respectively. With a positive a and a negative b , these formulae suggest that there may be a maximum time cost in activating a new batch. A smaller number of grid cells can reduce the cost, possibly because of the easiness of deciding if a QA round is completed. The curve shown in Figure 5(a) is $t_{new} = 958.0085 - \frac{1254.8737}{n_{batch}} + \frac{739.4750}{n_{batch}^2}$.

As shown in Figure 5(b), the t_{view} values estimated based on Equation (1) exhibit different patterns for the three application scenarios. This is understandable, as viewing and comprehending the three types of data objects may incur different time costs. In particular, viewing solar panel images is generally more time-consuming, except that the second data point seems to be an outlier. As shown in Figure 5(c), the estimated t_{single} values for JS-block and JS-grouping are similar. In contrast, the t_{single} values for SP-image are generally higher than those of JS-block and JS-grouping.

We thus separately model t_{view} and t_{single} for the three application scenarios. We choose to model t_{single} next, because we expect t_{single} to be less affected by the size of the data objects being displayed than t_{view} . The time cost to issue a single edit command, t_{single} , is mainly about the user's human motor control. The time cost to view a data object, t_{view} , requires the user's comprehension. Moreover, there are fewer single edit actions than viewing actions, and modeling t_{single} is expected to affect t_{view} less than modeling t_{view} before t_{single} .

Fitting t_{single} : Before modeling t_{single} , we reestimated the t_{single} data points by replacing the t_{new} variable in Equation (1) with modeled t_{new} values. The reestimated t_{single} data points for the three application scenarios are shown in Figure 5(d)–(f), respectively. Following the same procedure to gather and analyze candidate models, we have selected a constant model for each application scenario. The three models are:

- **For JS-block:** $t_{single} = 208.0818$
- **For JS-grouping:** $t_{single} = 193.8312$
- **For SP-image:** $t_{single} = 507.9517$

Fitting t_{view} : In the third iteration, we first reestimate the t_{view} data points using the modeled t_{new} and t_{single} data. As shown in Figure 5(g)–(i), the modeling of t_{new} and t_{single} hardly caused a noticeable displacement of the t_{view} data points. We can also easily observe a common phenomenon that the t_{view} values are higher when there are too many or too few grid cells (equivalently, when the sizes of grid cells are too small or too large). Following the same procedure for gathering and analyzing candidate models, we select the formula $y = a + bx + c/x$ for all three application scenarios as it semantically captures the effect of grid sizes with its second and third terms. The three models are:

- **For JS-block:** $t_{view} = 50.4283 + 4.3202n_{batch} + \frac{383.5033}{n_{batch}}$
- **For JS-grouping:** $t_{view} = 151.9951 + 2.2618n_{batch} + \frac{322.6178}{n_{batch}}$
- **For SP-image:** $t_{view} = 267.0109 + 7.9103n_{batch} + \frac{712.8504}{n_{batch}}$

Following the curve fitting and reestimation steps, we obtain the operator time costs in Table 4 in Appendix B.

5.3.4 Findings. Through the operator time cost estimations, we have the following observations:

- The initial estimations of the operator time costs exhibit low square errors as shown in Table 3 in Appendix B. The estimation is conducted with multiple linear regression, because we assume the user operations are sequential and the total time cost is a summation of the operator time costs. The low square errors imply that the assumption is reliable, at least for numeric modeling purposes.
- The operator time costs depend on layout and application, as shown in Figure 5. It implies that QA4ML interface designers need to optimize the layout to improve users' productivity.
- t_{view} follows a U-shaped trend with the number of grid cells in the grid layout (as shown in Figure 5(g)–(i)). We interpreted the trend as grid cells that are too large or too small are hard to view. The user may need to inspect small grid cells more carefully, which costs more time. For large grid cells, we interpret that the user may need additional pupil movements for the foveal vision to cover the visual representation of the data object.
- The estimations of t_{view} in SP-image are much larger than those in JS-block and JS-grouping. We conceive it is harder for the user to comprehend the aerial images in SP-image than the visual objects in JS-block and JS-grouping.

ALGORITHM 2: Grid-Based Interface (Single Edit)**Inputs:** true labels Y^* , default labels Y **Output:** user operation sequence $ops = (op_1, \dots, op_m)$

```

1:  $i \leftarrow 1$ 
2:  $ops \leftarrow []$ 
3: do
4:    $ops.push("new")$  ▷ request a new batch
5:    $batchEnd \leftarrow \min(i + n_{batch}, n)$ 
6:   for  $j \in (i, i + 1, \dots, batchEnd)$  do
7:      $ops.push("view")$  ▷ view an object
8:     if  $Y[j] \neq Y^*[j]$  then ▷ mislabeled?
9:        $ops.push("single")$  ▷ make a single edit
10:    end if
11:  end for
12:   $i = i + n_{batch}$ 
13: while  $i \leq n$  ▷ session ends?
14: return  $ops$ 

```

5.4 Simulations and Observations

In the following, we use simulation to examine the influence of application and layout on the total time cost $T_{session}$. We demonstrate using simulation to find the optimal layout of QA4ML interfaces.

5.4.1 Simulation Setup. We use simulations to estimate $T_{session}$ for different applications and layouts following the workflow in Figure 4. Algorithm 2 describes how to simulate an operation sequence in a simulation trial. We run the simulations with the following parameter values:

- **Interface layout:** all the *layout* for which we have estimated operator time costs in Section 5.3.
- **Dataset:** three conditions corresponding to the dataset size n and normalized confusion matrix cm of the three applications JS-block, JS-grouping, and SP-image introduced in Section 5.1.
- **Operator time costs:** t_{new} , t_{view} , and t_{single} to be the values measured for the given *application* and *layout* as shown in Figure 5 and Table 4 in Appendix B.

5.4.2 Result and Analysis. Figure 7 shows the simulation results in relation to different grid layouts and applications. The bar charts in the three applications all exhibit a U-shaped pattern. For JS-block and JS-grouping, the minimal total time cost for a QA session is achieved with the 4×5 layout. For SP-image, the minimum is achieved with the 3×4 layout.

The default labels have low error rates (1.4% for JS-block, 0.48% for JS-grouping, and 11.04% for SP-image). Thus, the total cost of single edit operations (T_{single}) is low compared with the total cost of the operations for starting a new batch (T_{new}) and the total cost of the viewing actions (T_{view}). The optimal layout reflects a tradeoff between T_{new} and T_{view} .

Layout significantly impacts $T_{session}$, as shown in Figure 7. The optimal layout depends on the QA application. No single layout is optimal for all the application scenarios. A design implication is that for QA4ML interfaces, UI specialists may need to optimize the layout separately for each application. Alternatively, QA4ML interfaces may provide the user with layout-switching functions.

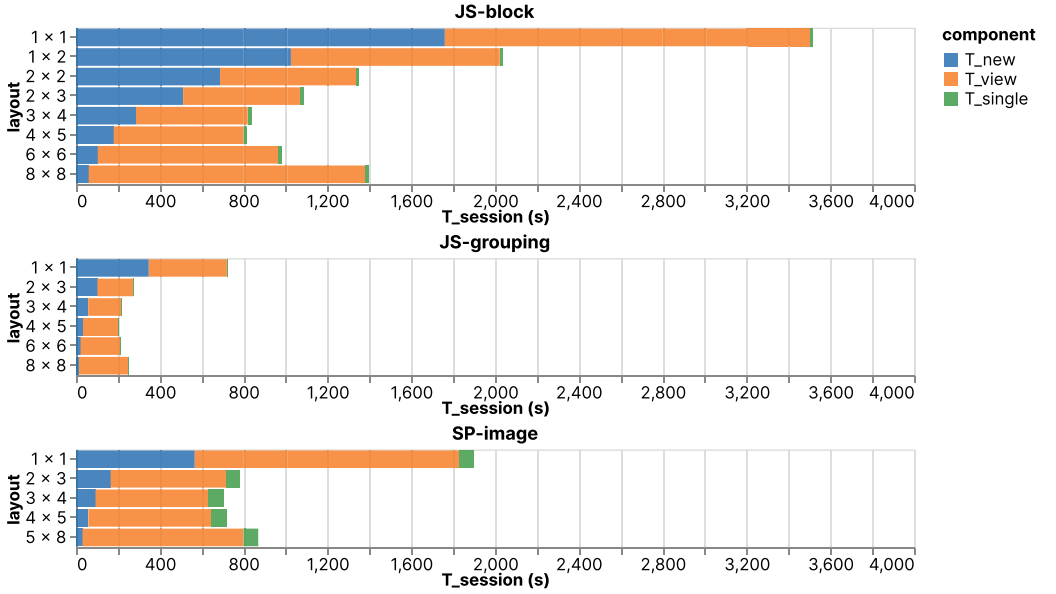


Fig. 7. Simulation of layouts: The effect of grid layout in the three applications. For each application, the total time cost follows a U-shaped curve. For JS-block and JS-grouping, the minimal is achieved at 4×5 . For SP-image, the minimal is achieved at 3×4 .

6 THE FACTOR OF USER LABEL STRATEGY

Section 5 has introduced the setting where using “single edit” commands is the only way to edit labels. For the grid-based interfaces, except for the 1×1 layout, all the other layouts present multiple data objects simultaneously. Thus, a natural extension of the interface functionality is to provide “batch edit” commands that simultaneously edit the label of multiple data objects. Users may use their intelligence to take shortcuts when batch edit commands are available.

This section uses simulation to examine the benefit of providing batch edit commands. When batch edit commands are available, the user has multiple methods to accomplish a QA session, such as using only the single edit commands (as is in Section 5) or combining single and batch edit commands. Thus, we model the user’s label strategy to choose between the methods.

The JS-block and JS-grouping applications concern active learning workflows, as illustrated in Figure 1(a). A user can dynamically decide when to stop the active learning after labeling k data objects interactively. If k is small, then the cost of manual labeling during active learning is low, but the ML model is likely less accurate, and more erroneous labels are to be corrected during QA4ML. However, the less accurate ML results are, the more benefit the batch edit commands may bring. Hence, it is interesting to simulate the impact of the user’s strategies for using batch edit commands in conjunction with different accuracy levels of the ML results.

6.1 Conceptual Model for Simulation

After introducing the batch edit commands, the user has multiple methods to carry out the QA processes. Moreover, the user may not use the same method for all the batches. Instead, the user may decide to apply different methods depending on the distribution of TP, TN, FP, and FN in each batch. In this section, we model the user’s strategy of choosing among the methods. The user strategy is essentially the “selection rules” in GOMS models [33].

6.1.1 Batch Edit Methods. Figure 3 extends the basic QA4ML workflow in Figure 4 with batch editing commands. For the quality assurance of binary classifications, we consider three batch edit commands:

- **All positive:** label the data objects “positive.”
- **All negative:** label the data objects “negative.”
- **Inverse all:** label the data objects to be the logical inverse of their current labels.

When quality-assuring the labels of a batch of data objects, the batch edit commands should be used at most once if a user makes no mistakes in observation and command selection. When the three batch edit commands are available, a user can observe all data objects in the editing panel and choose one of the following four methods to finish a round of quality assurance. Each method has a different time cost T_{round} .

- **Baseline (B):** using only “single edit” commands to correct all the false-positive/negative errors (number of single edits $N_{single} = K_{FP} + K_{FN}$). In this case, $T_{round} = T_0 + (K_{FP} + K_{FN}) \cdot t_{single}$.
- **Mostly positive (P):** issuing an “all positive” command, and then using “single edit” commands to correct the remaining errors ($N_{single} = K_{FP} + K_{TN}$). In this case, $T_{round} = T_0 + t_{allPositive} + (K_{FP} + K_{TN}) \cdot t_{single}$.
- **Mostly negative (N):** issuing an “all negative” command, and then using “single edit” commands to correct the remaining errors ($N_{single} = K_{FN} + K_{TP}$). In this case, $T_{round} = T_0 + t_{allNegative} + (K_{FN} + K_{TP}) \cdot t_{single}$.
- **Mostly wrong (W):** issuing an “inverse all” command, and then using “single edit” commands to correct the remaining errors ($N_{single} = K_{TP} + K_{TN}$). In this case, $T_{round} = T_0 + t_{inverseAll} + (K_{TP} + K_{TN}) \cdot t_{single}$.

In the time costs, $T_0 = t_{new} + t_{overview} + K \cdot t_{view}$. The operator time costs, $t_{overview}$, $t_{allPositive}$, $t_{allNegative}$, and $t_{inverseAll}$, are the time costs to overview all grid cells in the editing area and issue one of the three batch edit commands, respectively. Which of the four methods is the most efficient (with the minimal T_{round}) depends on the confusion matrix $\begin{pmatrix} K_{TN} & K_{FP} \\ K_{FN} & K_{TP} \end{pmatrix}$ of the batch of data objects.

The set of possible labeling methods depends on the available commands (denoted as $cmds$) for editing labels. Typically, “single edit” commands are available to ensure that each label can be edited individually. Thus, we assume $SingleEdit \in cmds$. When a batch edit command (“all positive,” “all negative,” or “inverse all”) is unavailable in the QA4ML interface, the corresponding labeling method (P, N, or W) is unavailable. For a QA4ML interface without batch edit commands, we refer to its $cmds$ as $cmds_{single} = \{SingleEdit\}$. When all four label edit commands are available, we refer to its $cmds$ as $cmds_{batch} = \{SingleEdit, AllPositive, AllNegative, InverseAll\}$.

6.1.2 Parameterizing User Label Strategy. The effectiveness of the label edit commands depends on a user’s ability to choose the right method that minimizes the time cost when facing a batch of data objects. We parameterize this factor of a user’s ability to make strategic choices so that we can simulate different levels of the user’s ability. The parameterization of user strategies corresponds to an implementation of “SelectEditCommand” in Algorithm 1. We introduce two model parameters as follows:

- **User skill level:** $u_{sl} \in [-1, 1]$ represents different levels of user skills, with 1 being the best and -1 being the worst.

- **User strategy uncertainty:** $u_{su} \in [0, 1]$ represents randomness in choosing among the methods, with 1 being entirely random and 0 being deterministic.

The overall idea of the parameterization is that the labeling methods can be ranked by time costs. A higher skill level corresponds to choosing a smaller rank index. The choice of a rank index may be associated with an error range. The strategy uncertainty captures the width of the error range.

When $u_{sl} = 0$, a user does not use batch edit commands and always chooses the baseline method B. When $u_{sl} = 1$, the user is highly experienced and always chooses the best method for each batch. Let N_m be the total number of methods. $N_m = 4$ when the batch edit commands are all available. $N_m = 1$ when only single edit commands are available. A simulation model knows the ground truth labels. Therefore, it can sort all labeling methods by time costs from 0 to $N_m - 1$, with the 0th being the best. The strategy of always choosing the 0th method corresponds to $u_{sl} = 1$. The strategy of always choosing the $(N_m - 1)$ -th method corresponds to $u_{sl} = -1$.

When $0 < u_{sl} < 1$, the simulation model chooses the k th labeling method between the best method at position 0 and the baseline method at position bl , such that $k = \text{round}((1 - u_{sl}) \cdot bl)$. If the baseline method is the best, then $bl = 0$ and $k = 0$.

When $-1 < u_{sl} < 0$, the simulation model chooses the k th labeling method between the baseline method at position bl and the worst method at position $(N_m - 1)$, such that $k = \text{round}(-u_{sl} \cdot (N_m - 1 - bl) + bl)$. Usually, users will not deliberately choose a batch edit command that would lead to more user operations than the baseline method. It is thus rare to have $u_{sl} < 0$.

When $u_{su} = 1$, the selection is entirely random among all N_m strategies. When $u_{su} = 0$, the selection of the labeling method is deterministic and is based on u_{sl} only. When $0 < u_{su} < 1$, the simulation model randomly chooses a labeling method ranked between the a th and b th positions. $a = \max(0, k - \delta)$, $b = \min(N_m - 1, k + \delta)$, and $\delta = \text{round}(u_{su} \cdot N_m)$.

6.2 Estimating Operator Time Costs

In principle, $t_{overview}$, $t_{allPositive}$, $t_{allNegative}$, and $t_{inverseAll}$ can be estimated in a manner similar to that for t_{new} , t_{view} , and t_{single} in Section 5. In the following, we consider a discount process for estimating these operator time costs.

It is intuitive to anticipate that $t_{allPositive}$, $t_{allNegative}$, and $t_{inverseAll}$ are close to t_{new} , because their corresponding buttons are all located at the header toolbar. We thus assume $t_{allPositive} = t_{allNegative} = t_{inverseAll} = t_{new}$. The frequency of using the commands “all positive,” “all negative,” and “inverse all” is much lower than other operations. Thus, even if the estimations of $t_{allPositive}$, $t_{allNegative}$, and $t_{inverseAll}$ have minor imprecisions, the imprecisions have a limited impact on the estimation of the total time cost.

The time that a user spends for overviewing all data objects ($t_{overview}$) before issuing (or skipping) a batch edit command can be costly when n_{batch} is large. It likely increases with n_{batch} as the user needs to comprehend more data objects. After the overview, there may be a small reduction in the time used for examining each individual data object (t_{view}). To capture these intuitions, in the following simulations, we assume a simple linear model $t_{overview} = n_{batch} \cdot 0.025$ with the unit being second to ensure $t_{overview}$ is monotonous with n_{batch} . We discount t_{view} by 0.0125 seconds when an overview is conducted.

6.3 Simulations and Observations

Through this set of simulations, we examine the influence of introducing batch edit commands and user’s label strategies to $T_{session}$.

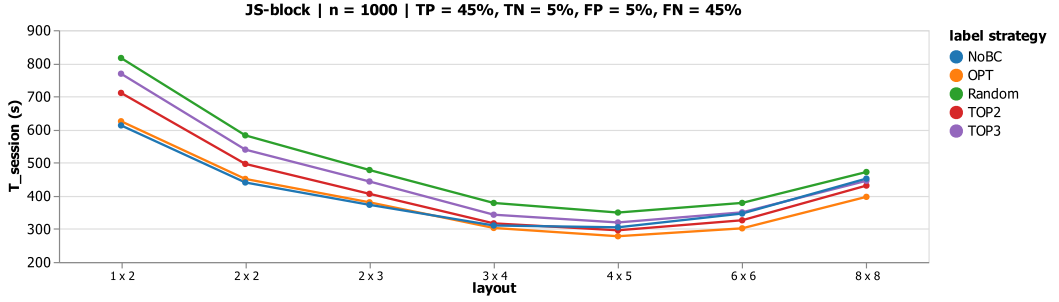


Fig. 8. Simulation of label strategies: The simulation result of the total time cost $T_{session}$ with five different label strategies: NoBC, OPT, Random, TOP2, and TOP3. The vertical axis starts from 200 instead of 0 to highlight the differences.

6.3.1 *Simulation Setup.* We run the simulations with the following parameter values:

- **Interface layout:** seven conditions of *layout* being $\{1 \times 2, 2 \times 2, 2 \times 3, 3 \times 4, 4 \times 5, 6 \times 6, 8 \times 8\}$.
- **Dataset:** $n = 1000$ and $cm = \begin{pmatrix} 0.05 & 0.05 \\ 0.45 & 0.45 \end{pmatrix}$.
- **Operator time costs:** t_{new} , t_{view} , $t_{overview}$, t_{single} , and t_{batch} to be the values measured for the JS-block application for any given *layout*.
- **Label strategy:** five conditions of $\langle cmds, u_{sl}, u_{su} \rangle$ corresponding to different label strategies
 - **NoBC:** never uses batch edit commands ($cmds = cmds_{single}$, $u_{sl} = 0$, $u_{su} = 0$).
 - **OPT:** choose optimal combinations of batch and single edit commands ($cmds = cmds_{batch}$, $u_{sl} = 1$, $u_{su} = 0$).
 - **Random:** choose a random combination of batch and single edit commands ($cmds = cmds_{batch}$, $u_{sl} = 1$, $u_{su} = 1$).
 - **TOP2:** choose randomly between the top 2 combinations of batch and single edit commands ($cmds = cmds_{batch}$, $u_{sl} = 1$, $u_{su} = 0.25$).
 - **TOP3:** choose randomly among the top 3 combinations of batch and single edit commands ($cmds = cmds_{batch}$, $u_{sl} = 1$, $u_{su} = 0.5$).

Ten repeated simulation trials with different random seeds are run for each combination of simulation parameters. The random seed determines the random permutation of data objects.

6.3.2 *Result and Analysis.* As shown in Figure 8, user's label strategy influences the time cost. For all the layouts with more grid cells than 2×3 , OPT is consistently better than NoBC. Note that NoBC can be better than OPT under some layouts. The interface for NoBC has no batch edit commands, and thus NoBC can waive the time cost to overview the data objects, which is required for the other label strategies. The minimal $T_{session}$ is achieved by OPT at 4×5 layout, which suggests that *introducing batch edit commands may save user effort*.

Similarly to Figure 7, the time cost follows a U-shaped curve. With the increase of n_{batch} , NoBC's relative performance consistently decreases compared with the other label strategies. The advantage of using batch edit commands increases with n_{batch} .

7 THE FACTOR OF DEFAULT LABEL ACCURACY

QA4ML workflows require the user to verify and correct machine predictions. While it is intuitive that accurate machine predictions save user effort, it is not straightforward to what extent

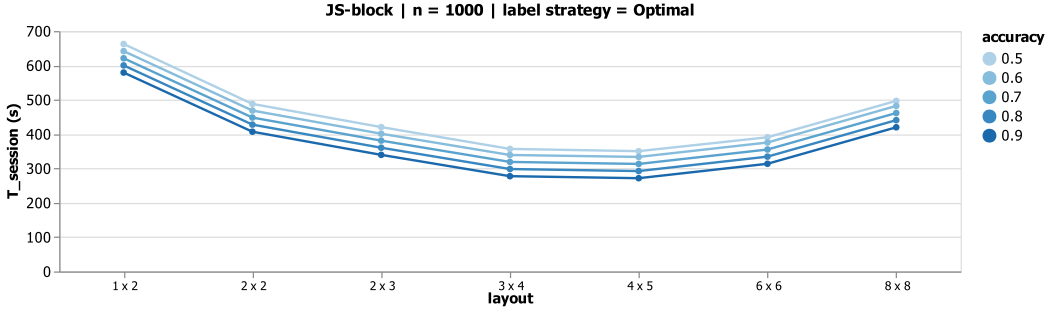


Fig. 9. Simulation of default label accuracies: The simulation result of the total time cost $T_{session}$ with five conditions of default label accuracy (0.5, 0.6, 0.7, 0.8, and 0.9).

accurate predictions reduce task completion time. This section investigates the impact of default label accuracy on the QA4ML time costs. In practice, UI specialists may conduct such simulations to examine how much time can be saved by improving machine predictions. In this way, UI specialists can decide whether it is worthwhile to improve the default label accuracy or whether it is more cost-effective to improve other aspects of the QA4ML interface.

7.1 Simulations and Observations

Through this set of simulations, we examine the influence of default label accuracy to $T_{session}$.

7.1.1 Simulation Setup. We run the simulations with the following parameter values:

- **Interface layout:** five conditions of *layout* being $\{1 \times 2, 2 \times 2, 2 \times 3, 3 \times 4, 4 \times 5, 5 \times 6, 8 \times 8\}$.
- **Dataset:** $n = 1000$ and five conditions of cm being $\{(\begin{smallmatrix} 0.5a & 0.5-0.5a \\ 0.5-0.5a & 0.5a \end{smallmatrix}) | a \in \{0.5, 0.6, 0.7, 0.8, 0.9\}\}$. The five conditions of cm correspond to different default label accuracies: 0.5, 0.6, 0.7, 0.8, 0.9.
- **Operator time costs:** t_{new} , t_{view} , $t_{overview}$, t_{single} , and t_{batch} to be the values measured for the JS-block application for any given *layout*.
- **Label strategy:** $cmds = cmds_{batch}$, $u_{sl} = 1$, and $u_{su} = 0$. With this configuration, the optimal label strategy is used for every batch.

Ten repeated simulation trials with different random seeds are run for each combination of simulation parameters. The random seed determines the random permutation of data objects and the assignment of default labels.

7.1.2 Result and Analysis. Figure 9 shows the simulation result. For all the layouts, *with the increase of default label accuracy, the total time cost consistently decreases*. Meanwhile, although accurate models save much time, using a good model (accuracy = 0.9) with a bad layout (1×2) may cost more time than using a bad model (accuracy = 0.5) with a good layout (3×4). This observation implies that *good interface and algorithm designs are both critical for saving user effort*.

8 THE FACTOR OF RANK METHOD

A general QA4ML workflow involves rank methods to reorder data objects, as shown in Figure 3. Rank methods can alter the local distributions of label categories and default labels' correctness in the batches. Rank methods may reduce the time cost by unbalancing the local confusion matrix

$\begin{pmatrix} K_{TN} & K_{FP} \\ K_{FN} & K_{TP} \end{pmatrix}$ of each batch. The effectiveness of rank methods may affect the opportunities of using batch edit commands and influence the QA4ML time cost.

One may use actual implementations of the rank methods for simulation. For example, Zhang et al. [61, 62] simulate the number of user operations in active learning workflows. Their simulations use the actual implementations of scoring functions in active learning to rank the data objects. Meanwhile, using actual implementations can be inefficient. The rank method needs to be implemented by the evaluator, which introduces an implementation cost. Additionally, the actual implementation can be computationally expensive and slow down the simulation.

In the following, we introduce a parameterization of rank methods. With the parameterization, we can simulate rank methods by configuring different parameter values. The parameterization approach also gives us more control over the characteristics of the rank methods in simulations.

8.1 Conceptual Model for Simulation

To capture the rank methods in the simulation model, it is desirable to model their ability to change the local confusion matrix in each batch. The modeling of the rank method corresponds to an implementation of “Rank” in Algorithm 1. The following focuses on the rank methods that can be modeled as selecting a subset of data objects according to some criteria and moving them to the front of the list of data objects. We refer to them as bipartition (“BiPart”) rank methods.

8.1.1 Parameterizing Rank Method. We parameterize a rank method with a matrix $rm \in [0, 1]_{c \times c}$ where c is the number of label categories. $rm_{i,j}$ denotes the rate of data objects with true label i and default label j that are moved to the front of the list by the rank method among all such data objects. The total number of data objects selected by the rank method rm and moved to the front of the list is thus $n_s = \sum_{i,j} rm_{i,j} \cdot n \cdot cm_{i,j}$. All the selected data objects are regarded as unordered. Thus, the selected data objects are randomly shuffled in the simulation. For binary classification, let $rm = \begin{pmatrix} p_{TN} & p_{FP} \\ p_{FN} & p_{TP} \end{pmatrix}$, we have $n_s = n_{TP}p_{TP} + n_{TN}p_{TN} + n_{FP}p_{FP} + n_{FN}p_{FN}$.

8.1.2 Examples. Consider a rank method with $rm = \begin{pmatrix} 0 & 0 \\ 1 & 1 \end{pmatrix}$ that moves data objects with ground truth labels positive (i.e., TPs and FNs) to the front of the list. For a batch containing a subset of these selected data objects, the user can use a batch edit command to set all their label categories to be “Positive.”

Consider another rank method with $rm = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}$ that selects FNs and FPs. Such a rank method moves all the data objects with incorrect default labels to the front, and all the user needs to do is to use a batch edit command to inverse all of their labels.

In these two examples, the rank methods are good as they create opportunities for the user to use the batch edit commands effectively. A rank method can make the distributions of TP, TN, FP, and FN less uniform. Any high concentration of one group of TP, TN, FP, or FN and combined groups of {TP, FN}, {TN, FP}, {TP, TN}, or {FP, FN} can be beneficial. In these situations, batch edit commands are beneficial. What is undesirable is any ordering result that groups a batch of data objects with similar numbers of TPs and FPs or similar numbers of TNs and FNs.

8.1.3 Quantifying Rank Method Performance. It is unrealistic to expect the rank method to be good enough to get all the TP/TN/FP/FN ranked top, as in the examples above. We quantify the performance of a rank method by its difference from an ideal selection. We denote an ideal selection with a binary matrix $target \in \{0, 1\}_{c \times c}$. In the matrix, $target_{i,j} = 1$ when data objects with true label i and default label j are intended to be selected.

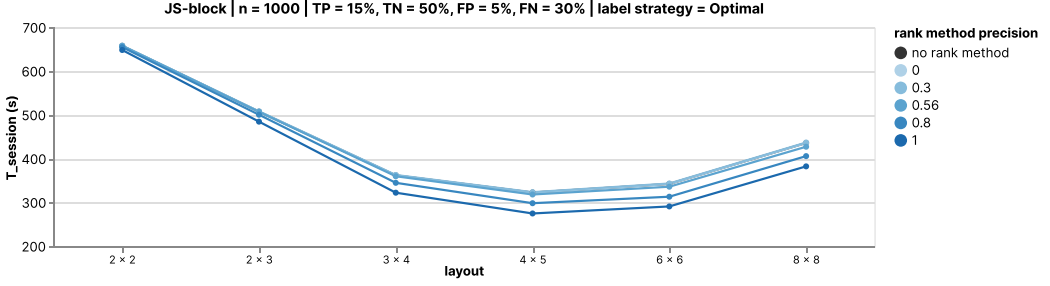


Fig. 10. Simulation of rank method precisions: Five conditions of rank method precision are simulated, including 0, 0.3, 0.56, 0.8, and 1. An additional condition called “no rank method” is simulated where the dataset is randomly ordered. The curves of “no rank method” and rank methods with precision 0 and 0.3 heavily overlap. The vertical axis starts from 200 instead of 0 to highlight the differences.

A rank method rm selects $n_s = \sum_{i,j} rm_{i,j} \cdot n \cdot cm_{i,j}$ data objects and moves them to the front of the list. As specified by $target$, among all the selected data objects, $\sum_{i,j} rm_{i,j} \cdot n \cdot cm_{i,j} \cdot target_{i,j}$ are intended to be selected. We use the rank method’s precision, $\frac{\sum_{i,j} rm_{i,j} \cdot cm_{i,j} \cdot target_{i,j}}{\sum_{i,j} rm_{i,j} \cdot cm_{i,j}}$, to quantify its performance. Note that the precision depends on not only the rank method rm itself but also the choice of the ideal selection $target$.

8.2 Simulations and Observations

Through this set of simulations, we examine the influence of introducing rank methods to $T_{session}$.

8.2.1 Simulation Setup. We run the simulations with the following parameter values:

- **Interface layout:** five conditions of $layout$ being $\{2 \times 2, 2 \times 3, 3 \times 4, 4 \times 5, 6 \times 6, 8 \times 8\}$.
- **Dataset:** $n = 1000$ and $cm = \begin{pmatrix} 0.50 & 0.05 \\ 0.30 & 0.15 \end{pmatrix}$.
- **Operator time costs:** t_{new} , t_{view} , $t_{overview}$, t_{single} , and t_{batch} to be the values measured for the JS-block application for any given $layout$.
- **Label strategy:** $cmds = cmds_{batch}$, $u_{sl} = 1$, and $u_{su} = 0$. With this configuration, the optimal label strategy is used for every batch.
- **Rank method:** five conditions of rm being $\{(\frac{1-a}{3}, \frac{1-a}{3}) | a \in \{0, 0.25, 0.5, 0.75, 1\}\}$. We have also simulated a scenario where no rank method is used, and the dataset is randomly ordered.

We set $target = \begin{pmatrix} 0 & 0 \\ 1 & 0 \end{pmatrix}$, i.e., the rank method’s goal is to select false-negatives. Thus, given $cm = \begin{pmatrix} 0.50 & 0.05 \\ 0.30 & 0.15 \end{pmatrix}$, the BiPart rank method’s precision is $\frac{9a}{2a+7}$. When a equals 0, 0.25, 0.5, 0.75, and 1, the corresponding rank method precisions are (approximately) 0, 0.3, 0.56, 0.8, and 1.

Ten repeated simulation trials with different random seeds are run for each combination of simulation parameters. The random seed determines the ordering of data objects and the assignment of default labels.

8.2.2 Result and Analysis. Figure 10 shows the simulation result. Compared with not using rank methods, introducing the rank methods reduces the time cost. For all the layouts, *with the increase of rank method precision, the total time cost consistently decreases*. Besides, the benefit of having rank methods with high precision is more prominent for layouts with larger n_{batch} . For a larger n_{batch} , more data labels can be edited by activating one batch edit command. The result provides

Table 1. The Parameters of Our Simulation Model

Parameter	Data Type	Estimation	Description
n	$\mathbb{N}^+$	external	The number of data objects.
n_{batch}	$\mathbb{N}^+$	observed	The number of data objects of each batch.
cm	$[0, 1]_{c \times c}$	external	The confusion matrix of default labels for the data to be quality-assured.
t_{new}	$\mathbb{R}^+$	captured	The average time cost to activate the “new batch” command to fetch the next batch of data objects.
t_{view}	$\mathbb{R}^+$	captured	The average time cost to view a data object and its label in a grid cell in the editing panel (including the cost of viewing the context panel if necessary).
$t_{overview}$	$\mathbb{R}^+$	captured	The average time cost to overview all data objects in the editing panel (including the cost of viewing the context panel if necessary).
t_{single}	$\mathbb{R}^+$	captured	The average time cost to activate a command to change the label of a data object in the editing panel.
t_{batch}	$\mathbb{R}^+$	captured	The average time cost to activate a batch edit command to change the labels of all the data objects in the editing panel.
u_{sl}	$[-1, 1]$	observed	The user skill level in using batch edit commands.
u_{su}	$[0, 1]$	observed	The user strategy uncertainty in using batch edit commands.
$cmds$	$\text{set}\langle\text{string}\rangle$	observed	The set of UI commands.
rm	$[0, 1]_{c \times c}$	observed	The rank method parameterized by a matrix.

We use standard math notations to denote the data types, such as $\mathbb{N}$ for integers, $\mathbb{R}$ for real numbers, $+$ for > 0 , and $[a, b]$ for a range.

the design implication that for QA4ML interfaces, *it is beneficial to integrate rank methods with high precision to promote the use of batch edit commands.*

9 SUMMARY OF THE SIMULATION MODEL

The simulation model described above is intended to be used by UI specialists. The model can be used for different applications by specifying different simulation parameter values. Table 1 summarizes all the simulation parameters discussed in the previous sections.

Some parameters are independent of the QA4ML process and are marked “external.” For example, before the QA4ML process, there should be a process of applying an ML model to generate a collection of labeled data objects to be checked by the QA4ML process. Hence, the total number of data objects (n) and the confusion matrix (cm) of the ML predictions should be known or estimable to UI specialists who conduct the simulation.

UI specialists who are to optimize a QA4ML UI typically have some knowledge about the users who use the UI to perform the QA4ML tasks and intuitions on what design may be appropriate. We use “observed” to categorize parameters to be defined by UI specialists based on their observation of the data objects, users, tasks, and available rank methods and their likely effectiveness in the application context. UI specialists do not need to fix any parameters to a single value or setting. The benefit of using simulation is that UI specialists can consider many options.

For example, given some example data objects to be quality-assured, UI specialists can normally judge whether the display size of the data objects in the editing panel is too small or too large. Meanwhile, UI specialists may be uncertain whether a 4×5 layout is better than a 5×7 layout. UI specialists can simulate both scenarios by assigning different values to the parameter for batch size

(n_{batch}). Similarly, UI specialists may consider having users with different QA4ML experiences and at different skill levels. UI specialists can simulate different options by assigning different values to those user-related parameters (u_{sl} and u_{su}). Choosing parameter values related to the rank methods requires more technical knowledge. UI specialists may select a few relatively effective rank methods and determine their selection matrices (rm).

Table 1 lists the operator time costs. Some of them, such as t_{new} , are relatively generic. They are relatively easy to measure and can be reused across different QA4ML applications. Others, such as t_{view} , are highly application dependent and need to be modeled for each application separately. We recommend that UI specialists conduct experiments to capture a few trials of using a QA4ML UI. The timing parameters can be estimated based on captured data one by one, starting from simple ones, such as t_{new} .

While we focus on the quality assurance of classification labels, this approach can be adapted to other quality assurance tasks for model predictions (e.g., point cloud segmentation, time series forecasting) as long as the user routine can be modeled.

10 DISCUSSION

Our simulation approach provides an alternative to efficiently evaluate and compare design variations of quality assurance interfaces. As our approach can be seen as applying the Keystroke-Level Model to quality assurance tasks in grid-based interfaces, we bear the limitations of the Keystroke-Level Model. The following reflects on the limitations of our approach and future work directions.

Relaxing assumptions: Our modeling utilizes several assumptions, including that the user makes no mistakes, the machine is fast enough, and the user operations are sequential. Future work may extend the simulation models to relax these assumptions. By assuming the user makes no mistakes in carrying out the operations, we estimate a lower bound of the task completion time. In the future, one may model the time cost caused by users' erroneous use of commands in the interface and the effort to correct such errors. Latency of the machine operations and parallelized human and machine operations may be considered when estimating the task completion time. Modeling other factors may also improve the task completion time's estimation accuracy, such as user fatigue that leads to time-varying performance.

Using evaluation metrics other than task completion time: This work focuses on the evaluation metric of task completion time. The reason is that QA4ML processes are typically laborious for the user, making the task completion time a critical metric in this scenario. The goal of quality assurance is well-defined, i.e., fixing incorrect labels, making it feasible to estimate the task completion time. There are other crucial criteria in evaluating user interfaces, such as user satisfaction and difficulties in learning the UI. Prior work has studied using model-based evaluation for criteria beyond task completion time, such as using GOMS to evaluate learning time [33]. Future work may examine how to adapt prior work to QA4ML.

Simulating interface designs beyond the grid-based template: Our modeling mainly concerns the grid-based interfaces. These interfaces are based on buttons and menus, and the number of states of the interface is not too many. For QA4ML interfaces with interaction mechanisms beyond button-based interactions, such as a paint-based interface to quality-assure image mask predictions, the modeling and simulation of user operations will need more considerations.

Validating with more experiment trials from different users: In modeling operator time costs in Section 5.3, we use the experiment log from self-experimentation to demonstrate how to do the modeling. The implications of simulation results, such as the optimal layout, can be validated through future experiment logs. By gathering data from multiple users, we may investigate additional user-related aspects, such as the variation of the operator time costs among users. Note that future experiments are for accumulating records to validate our simulation model's

prediction accuracy on the task completion time. In production scenarios, it is unnecessary and undesirable for evaluators using our model to use user studies to validate all the simulation results. This would diminish the purpose of using model-based evaluation as an efficient evaluation approach.

Improving the simulation approach as a continuous process: After optimizing a user interface with the simulation approach and deploying it in practice, it is possible to collect deployment data and compare them with the predictions made by a simulation model. The comparison may enable the identification of causes of any inconsistency between model predictions and empirical data, therefore facilitating model improvements. Although many QA4ML user interfaces may be single-use (e.g., for a specific QA4ML workflow), the insights obtained may still enhance the simulation approach for optimizing other QA4ML interfaces in the future.

Evaluating the overall cost-benefit of the simulation approach: As a medium-to-long-term research agenda, it is highly desirable to collect deployment data from different QA4ML applications to evaluate the overall cost-benefit of using the simulation approach to optimize QA4ML interfaces. Such a meta-evaluation [49, 54], which has been used in prior HCI research [28, 42], will allow us to obtain more general findings that apply to a broad range of applications where traditional user studies may be too difficult or costly to conduct.

11 CONCLUSION

In quality-sensitive applications of QA4ML, the user needs to use QA4ML interfaces intensively. In such scenarios, interface evaluation and optimization bring significant benefits in terms of saved time for the user.

This work uses model-based evaluation to assess and optimize QA4ML interface design parameters through simulations. We focus on evaluating the time costs in QA4ML as QA4ML is typically labor-intensive. Our approach encompasses modeling the user's routine operations in QA4ML workflows, the user's operation time costs, the algorithmic assistance provided by the interface, and the user's strategy of using the interface with multiple commands available. We use data-driven modeling to estimate data- and interface-dependent operation time costs. We demonstrate that through modeling and simulating the task completion time, QA4ML interfaces can be evaluated and optimized.

Using QA4ML application scenarios, including data extraction from historical visualizations, we demonstrate the need for such an approach and the practical feasibility of using simulation to optimize the interface design. We demonstrate the influence of various factors on the total time cost of QA4ML, including interface layout, application scenario, availability of interface functions, user's label strategy, default label accuracy, and rank method's precision. The simulations have derived various findings, such as the dependency of the optimal layout condition on the application.

Model-based evaluation cannot replace user-centered approaches in designing and evaluating interfaces. It is impractical to expect the modeling approach to capture all the aspects of human factors. For ill-defined user tasks, such as open-ended data exploration, the evaluator may be unable to model the user's routine operations, making the simulation infeasible.

When the user routine can be modeled, model-based evaluation is a cost-effective alternative for UI specialists to evaluate and optimize the interface. Simulation makes it possible to swiftly explore potential design parameters of interest and identify the influence of various factors. It enables fast prototyping and iteration in early-stage interface development before expensive and time-consuming user studies are conducted. With the simulations that can address many small but potentially costly design issues, we can focus our engagement with users on big design questions about data, ML models, workflows, and application contexts.

APPENDICES

Two appendices accompany this work. The first appendix is titled “Estimating Operator Time Costs” (Appendix A). The second appendix is titled “Modeling and Reestimating Operator Time Costs” (Appendix B). Both appendices are included in the supplementary materials.

ACKNOWLEDGMENTS

We want to express our gratitude to the people who facilitated this project, particularly Dolores Romero Morales from Copenhagen Business School.

REFERENCES

- [1] Johnny Accot and Shumin Zhai. 1997. Beyond fitts’ law: Models for trajectory-based HCI tasks. In *Proceedings of the SIGCHI Conference on Human Factors in Computing Systems (CHI’97)*. ACM, New York, NY, 295–302.
- [2] Bilal Alsallakh, Allan Hanbury, Helwig Hauser, Silvia Miksch, and Andreas Rauber. 2014. Visual methods for analyzing probabilistic classification data. *IEEE Trans. Visual. Comput. Graph.* 20, 12 (Dec. 2014), 1703–1712.
- [3] John R. Anderson. 1983. *The Architecture of Cognition*. Harvard University Press, Mahwah, NJ.
- [4] John R. Anderson. 1993. *Rules of the Mind*. Lawrence Erlbaum Associates, Mahwah, NJ.
- [5] Keith Andrews. 2008. Evaluation comes in many guises. In *Proceedings of the CHI Workshop on Beyond Time and Errors: Novel Evaluation Methods for Information Visualization (BELIV’08)*. 7–8.
- [6] Mykhaylo Andriluka, Jasper R. R. Uijlings, and Vittorio Ferrari. 2018. Fluid annotation: A human-machine collaboration interface for full image annotation. In *Proceedings of the ACM International Conference on Multimedia (MM’18)*. ACM, New York, NY, 1957–1966.
- [7] Leif Azzopardi. 2011. The economics in interactive information retrieval. In *Proceedings of the International ACM SIGIR Conference on Research and Development in Information Retrieval (SIGIR’11)*. ACM, New York, NY, 15–24.
- [8] Leif Azzopardi, Diane Kelly, and Kathy Brennan. 2013. How query cost affects search behavior. In *Proceedings of the International ACM SIGIR Conference on Research and Development in Information Retrieval (SIGIR’13)*. ACM, New York, NY, 23–32.
- [9] Louise Barkhuus and Jennifer A. Rode. 2007. From mice to men—24 years of evaluation in CHI. In *Proceedings of the SIGCHI Conference on Human Factors in Computing Systems (CHI’07)*. ACM, New York, NY, 16.
- [10] A. Bäuerle, H. Neumann, and T. Ropinski. 2020. Classifier-guided visual correction of noisy labels for image classification tasks. *Comput. Graph. Forum* 39, 3 (June 2020), 195–205.
- [11] David V. Beard, Dana K. Smith, and Kevin M. Denelsbeck. 1996. Quick and dirty GOMS: A case study of computed tomography interpretation. *Hum.-Comput. Interact.* 11, 2 (June 1996), 157–180.
- [12] Maryam Booshehrian, Torsten Möller, Randall M. Peterman, and Tamara Munzner. 2012. Vismon: Facilitating analysis of trade-offs, uncertainty, and sensitivity in fisheries management decision making. *Comput. Graph. Forum* 31, 3 (June 2012), 1235–1244.
- [13] N. Boukhelifa, A. Bezerianos, R. Chang, C. Collins, S. Drucker, A. Endert, J. Hullman, C. North, M. Sedlmair, and Theresa-Marie Rhyne. 2020. Challenges in evaluating interactive visual machine learning systems. *IEEE Comput. Graph. Appl.* 40, 6 (2020), 88–96.
- [14] Nicholas J. Bryan, Gautham J. Mysore, and Ge Wang. 2014. ISSE: An interactive source separation editor. In *Proceedings of the SIGCHI Conference on Human Factors in Computing Systems (CHI’14)*. ACM, New York, NY, 257–266.
- [15] Stuart K. Card, Thomas P. Moran, and Allen Newell. 1980. The keystroke-level model for user performance time with interactive systems. *Commun. ACM* 23, 7 (July 1980), 396–410.
- [16] Duen Horng Chau, Aniket Kittur, Jason I. Hong, and Christos Faloutsos. 2011. Apollo: Making sense of large network data by combining rich user interaction and machine learning. In *Proceedings of the SIGCHI Conference on Human Factors in Computing Systems (CHI’11)*. ACM, New York, NY, 167–176.
- [17] Changjian Chen, ZhaoWei Wang, Jing Wu, Xiting Wang, Lan-Zhe Guo, Yu-Feng Li, and Shixia Liu. 2021. Interactive graph construction for graph-based semi-supervised learning. *IEEE Trans. Vis. Comput. Graph.* 27, 9 (2021), 3701–3716.
- [18] Quan Ze Chen, Tobias Schnabel, Besmira Nushi, and Saleema Amershi. 2022. HINT: Integration testing for AI-based features with humans in the loop. In *Proceedings of the International Conference on Intelligent User Interfaces (IUI’22)*. ACM, New York, NY, 549–565.
- [19] Gerald P. Chubb. 1981. SAINT, A digital simulation language for the study of manned systems. In *Manned Systems Design*, Jan Moraal and Karl-Friedrich Kraiss (Eds.). Plenum Press, New York, NY, 153–179.
- [20] Jingyu Cui, Fang Wen, Rong Xiao, Yuandong Tian, and Xiaou Tang. 2007. EasyAlbum: An interactive photo annotation system based on face clustering and re-ranking. In *Proceedings of the SIGCHI Conference on Human Factors in Computing Systems (CHI’07)*. ACM, New York, NY, 367–376.

- [21] Alan Dix, Janet Finlay, Gregory D. Abowd, and Russell Beale. 2004. *Human-Computer Interaction*. Pearson, London, UK.
- [22] John J. Dudley and Per Ola Kristensson. 2018. A review of user interface design for interactive machine learning. *ACM Trans. Interact. Intell. Syst.* 8, 2, Article 8 (June 2018), 37 pages.
- [23] Jerry Alan Fails and Dan R. Olsen. 2003. Interactive machine learning. In *Proceedings of the International Conference on Intelligent User Interfaces (IUI'03)*. ACM, New York, NY, 39–45.
- [24] Paul M. Fitts. 1992. The information capacity of the human motor system in controlling the amplitude of movement. *J. Exp. Psychol.: Gen.* 121, 3 (1992), 262–269.
- [25] James Fogarty, Desney Tan, Ashish Kapoor, and Simon Winder. 2008. CueFlik: Interactive concept learning in image search. In *Proceedings of the SIGCHI Conference on Human Factors in Computing Systems (CHI'08)*. ACM, New York, NY, 29–38.
- [26] Krzysztof Gajos and Daniel S. Weld. 2004. SUPPLE: Automatically generating user interfaces. In *Proceedings of the International Conference on Intelligent User Interfaces (IUI'04)*. ACM, New York, NY, 93–100.
- [27] Humberto S. Garcia Caballero, Michel A. Westenberg, Binyam Gebre, and Jarke J. van Wijk. 2019. V-Awake: A visual analytics approach for correcting sleep predictions from deep learning models. *Comput. Graph. Forum* 38, 3 (2019), 1–12.
- [28] Richard Gong and David Kieras. 1994. A validation of the GOMS model methodology in the development of a specialized, commercial software application. In *Proceedings of the SIGCHI Conference on Human Factors in Computing Systems (CHI'94)*. ACM, New York, NY, 351–357.
- [29] Wayne D. Gray and Marilyn C. Salzman. 1998. Damaged merchandise? A review of experiments that compare usability evaluation methods. *Hum.-Comput. Interact.* 13, 3 (Sept. 1998), 203–261.
- [30] Yves Guiard, Michel Beaudouin-Lafon, Julien Bastin, Dennis Pasveer, and Shumin Zhai. 2004. View size and pointing difficulty in multi-scale navigation. In *Proceedings of the Working Conference on Advanced Visual Interfaces (AVI'04)*. ACM, New York, NY, 117–124.
- [31] Benjamin Höferlin, Rudolf Netzel, Markus Höferlin, Daniel Weiskopf, and Gunther Heidemann. 2012. Inter-active learning of ad-hoc classifiers for video visual analytics. In *Proceedings of the IEEE Conference on Visual Analytics Science and Technology (VAST'12)*. IEEE, Los Alamitos, CA, 23–32.
- [32] Thomas S. Huang, Charlie K. Dagli, Shyamsundar Rajaram, Edward Y. Chang, Michael I. Mandel, Graham E. Poliner, and Daniel P. W. Ellis. 2008. Active learning for interactive multimedia retrieval. *Proc. IEEE* 96, 4 (April 2008), 648–667.
- [33] Bonnie E. John and David E. Kieras. 1996. The GOMS family of user interface analysis techniques: Comparison and contrast. *ACM Trans. Comput.-Hum. Interact.* 3, 4 (Dec. 1996), 320–351.
- [34] Bonnie E. John and David E. Kieras. 1996. Using GOMS for user interface design and evaluation: Which technique? *ACM Trans. Comput.-Hum. Interact.* 3, 4 (Dec. 1996), 287–319.
- [35] Bonnie E. John and Alonso H. Vera. 1992. A GOMS analysis of a graphic machine-paced, highly interactive task. In *Proceedings of the SIGCHI Conference on Human Factors in Computing Systems (CHI'92)*. ACM, New York, NY, 251–258.
- [36] Roli Khanna, Jonathan Dodge, Andrew Anderson, Rupika Dikkala, Jed Irvine, Zeyad Shureih, Kin-Ho Lam, Caleb R. Matthews, Zhengxian Lin, Minsuk Kahng, Alan Fern, and Margaret Burnett. 2022. Finding AI's faults with AAR/AI: An empirical study. *ACM Trans. Interact. Intell. Syst.* 12, 1, Article 1 (March 2022), 33 pages.
- [37] David E. Kieras. 1988. Towards a practical GOMS model methodology for user interface design. In *Handbook of Human-Computer Interaction*, Martin G. Helander (Ed.). Elsevier, Amsterdam, The Netherlands, 135–157.
- [38] David E. Kieras and Anthony J. Hornof. 2014. Towards accurate and practical predictive models of active-vision-based visual search. In *Proceedings of the SIGCHI Conference on Human Factors in Computing Systems (CHI'14)*. ACM, New York, NY, 3875–3884.
- [39] David E. Kieras and Thomas P. Santoro. 2004. Computational GOMS modeling of a complex team task: Lessons learned. In *Proceedings of the SIGCHI Conference on Human Factors in Computing Systems (CHI'04)*. ACM, New York, NY, 97–104.
- [40] David E. Kieras, Scott D. Wood, and David E. Meyer. 1997. Predictive engineering models based on the EPIC architecture for a multimodal high-performance human-computer interaction task. *ACM Trans. Comput.-Hum. Interact.* 4, 3 (Sept. 1997), 230–275.
- [41] Karl-Friedrich Kraiss. 1981. A display design and evaluation study using task network models. *IEEE Trans. Syst. Man Cybernet.* 11, 5 (May 1981), 339–351.
- [42] Clayton Lewis, Peter G. Polson, Cathleen Wharton, and John Rieman. 1990. Testing a walkthrough methodology for theory-based design of walk-up-and-use interfaces. In *Proceedings of the SIGCHI Conference on Human Factors in Computing Systems (CHI'90)*. ACM, New York, NY, 235–242.
- [43] Clayton Lewis and John Rieman. 1993. *Task-Centered User Interface Design*.

- [44] Shixia Liu, Changjian Chen, Yafeng Lu, Fangxin Ouyang, and Bin Wang. 2019. An interactive method to improve crowdsourced annotations. *IEEE Trans. Vis. Comput. Graph.* 25, 1 (Jan. 2019), 235–245.
- [45] I. Scott MacKenzie and William Buxton. 1992. Extending fitts' law to two-dimensional tasks. In *Proceedings of the SIGCHI Conference on Human Factors in Computing Systems (CHI'92)*. ACM, New York, NY, 219–226.
- [46] Jakob Nielsen and Rolf Molich. 1990. Heuristic evaluation of user interfaces. In *Proceedings of the SIGCHI Conference on Human Factors in Computing Systems (CHI'90)*. ACM, New York, NY, 249–256.
- [47] Antti Oulasvirta, Per Ola Kristensson, Xiaojun Bi, and Andrew Howes (Eds.). 2018. *Computational Interaction*. Oxford University Press, Oxford, UK.
- [48] Anjana Ramkumar, Pieter Jan Stappers, Wiro J. Niessen, Sonja Adebahr, Tanja Schimek-Jasch, Ursula Nestle, and Yu Song. 2017. Using GOMS and NASA-TLX to evaluate human-computer interaction process in interactive segmentation. *Int. J. Hum.-Comput. Interact.* 33, 2 (2017), 123–134.
- [49] Michael Scriven. 1969. An introduction to meta-evaluation. *Educ. Prod. Rep.* 2, 5 (Feb. 1969), 36–38.
- [50] Andrew Sears and Julie A. Jacko (Eds.). 2007. *The Human-Computer Interaction Handbook: Fundamentals, Evolving Technologies and Emerging Applications*. CRC Press, Boca Raton, FL.
- [51] Burr Settles, Mark Craven, and Lewis Friedland. 2008. Active learning with real annotation costs. In *Proceedings of the NIPS Workshop on Cost-Sensitive Learning*. 1–10.
- [52] John Snow. 1855. *On the Mode of Communication of Cholera*. John Churchill, London, UK.
- [53] Thilo Spinner, Udo Schlegel, Hanna Schäfer, and Mennatallah El-Assady. 2020. explAiner: A visual analytics framework for interactive and explainable machine learning. *IEEE Trans. Vis. Comput. Graph.* 26, 1 (2020), 1064–1074.
- [54] Daniel L. Stufflebeam. 1978. Meta evaluation: An overview. *Eval. Health Profess.* 1, 1 (April 1978), 17–43.
- [55] Bongwon Suh and Benjamin B. Bederson. 2007. Semi-automatic photo annotation strategies using event based clustering and clothing based person recognition. *Interact. Comput.* 19, 4 (July 2007), 524–544.
- [56] Jinhui Tang, Qiang Chen, Meng Wang, Shuicheng Yan, Tat-Seng Chua, and Ramesh Jain. 2013. Towards optimizing human labeling for interactive image tagging. *ACM Trans. Multimedia Comput. Commun. Appl.* 9, 4, Article 29 (Aug. 2013), 18 pages.
- [57] Kashyap Todi, Gilles Bailly, Luis Leiva, and Antti Oulasvirta. 2021. Adapting user interfaces with model-based reinforcement learning. In *Proceedings of the SIGCHI Conference on Human Factors in Computing Systems (CHI'21)*. ACM, New York, NY, USA, Article 573, 13 pages.
- [58] Jiachen Wang, Kejian Zhao, Dazhen Deng, Anqi Cao, Xiao Xie, Zheng Zhou, Hui Zhang, and Yingcai Wu. 2020. Tac-simur: Tactic-based simulative visual analytics of table tennis. *IEEE Trans. Vis. Comput. Graph.* 26, 1 (Jan. 2020), 407–417.
- [59] Yingcai Wu, Ji Lan, Xinhuan Shu, Chenyang Ji, Kejian Zhao, Jiachen Wang, and Hui Zhang. 2018. iTTVis: Interactive visualization of table tennis data. *IEEE Trans. Vis. Comput. Graph.* 24, 1 (Jan. 2018), 709–718.
- [60] Shouxing Xiang, Xi Ye, Jiazhi Xia, Jing Wu, Yang Chen, and Shixia Liu. 2019. Interactive correction of mislabeled training data. In *Proceedings of the IEEE Conference on Visual Analytics Science and Technology (VAST'19)*. IEEE, Los Alamitos, CA, 57–68.
- [61] Yu Zhang, Bob Coecke, and Min Chen. 2019. On the cost of interactions in interactive visual machine learning. In *Proceedings of the IEEE VIS Workshop on Evaluation of Interactive Visual Machine Learning Systems*. IEEE, Los Alamitos, CA, 5.
- [62] Yu Zhang, Bob Coecke, and Min Chen. 2021. MI3: Machine-initiated intelligent interaction for interactive classification and data reconstruction. *ACM Trans. Interact. Intell. Syst.* 11, 3–4, Article 18 (Aug. 2021), 34 pages.
- [63] Yu Zhang, Yun Wang, Haidong Zhang, Bin Zhu, Siming Chen, and Dongmei Zhang. 2022. OneLabeler: A flexible system for building data labeling tools. In *Proceedings of the SIGCHI Conference on Human Factors in Computing Systems (CHI'22)*. ACM, New York, NY.
- [64] Yinan Zhang and Chengxiang Zhai. 2015. Information retrieval as card playing: A formal model for optimizing interactive retrieval interface. In *Proceedings of the International ACM SIGIR Conference on Research and Development in Information Retrieval (SIGIR'15)*. ACM, New York, NY, 685–694.

Received 25 October 2022; revised 26 February 2023; accepted 2 April 2023